

Job to Invoice Product Requirements Document

Developer handoff and release specification

Version 1.0 | 11 September 2026 | Prepared for Ahmed Waleed Bajwa and the implementation team

Build objective

Create an iPhone app and secure customer approval website for solo service businesses. Keep the original quote, approved extra work and final invoice consistent, with reliable recovery and complete financial history.

Use this as the shared product, engineering and QA contract. It includes launch decisions, 28 screen specifications, database and API contracts, 12 financial fixtures, 68 acceptance scenarios, and operational release gates. Proposed requirements describe the app to be built; they are not claims that implementation or validation is already complete.

Launch baseline

US market • English • USD • One owner per business • iPhone first • Browser-based customer review • Manual recording of externally handled payments

Read first

Sections 01 to 06 define scope and invariants. Sections 07 to 18 define product behaviour. Sections 19 to 27 define implementation and safeguards. Sections 28 to 36 define testing, release and handover. Section 34 lists the owner-supplied production inputs.

Contents

01 Purpose and document authority
02 Release decisions and boundaries
03 Product outcomes and validation
04 Roles and permissions
05 Domain vocabulary and invariants
06 End to end journeys
07 Navigation and visual specification
08 Screen inventory and field contracts
09 Accounts customers and catalogue
10 Quote and job lifecycle
11 Change orders and reductions
12 Approval security and evidence
13 Monetary calculations and fixtures
14 Invoices credits payments and corrections
15 Subscription and entitlement contract
16 Draft persistence and synchronization
17 Documents photos and exports
18 Communications and exact transactional copy
19 Reference architecture and repository
20 Database schema contract
21 API conventions and payloads
22 Endpoint inventory
23 Transaction algorithms
24 Security and abuse controls
25 Privacy retention and account deletion
26 Accessibility performance and resilience
27 Analytics instrumentation
28 Acceptance test matrix
29 Developer delivery and definition of done
30 Build sequence and engineering gates
31 Operations monitoring and incident response
32 Store submission and release acceptance
33 Configuration and service provisioning
34 Business inputs and decision register
35 Primary sources and accuracy boundaries
36 Handover summary and traceability

01 Purpose and document authority

Build an iPhone application that lets a solo residential service operator create a quote, obtain approval for additional work, and issue an invoice containing the approved charges. Customers review and approve through a secure mobile website without installing an app. This document specifies the first production release, including product behaviour, interface requirements, calculation rules, persistence, APIs, security, operations and acceptance tests.

The central promise is “Keep every approved extra on the final invoice.” The first audience is US sole proprietors and small owner-operated handyman businesses. This is the selected launch assumption from the preceding opportunity analysis, not a validated claim that all handymen need this product. Validate the workflow with real users while implementing it. The working title Job to Invoice is descriptive, not a cleared brand.

MUST means a release requirement. SHOULD means a post-launch improvement unless explicitly included in the release matrix. MAY means an implementation option that cannot weaken a MUST. Requirements carry IDs so designers, engineers and QA can trace delivery. The product rules in this document prevail over illustrative copy and examples; financial invariants and authorization rules prevail over screen shortcuts. Resolve a true conflict by recording a numbered decision and updating this document, rather than silently picking a behaviour.

This is a build specification, not existing software or a representation that production credentials, legal terms, user validation or store approval already exist. The engineering team must supply source code, executable API schemas, migrations, automated tests and operational configuration as implementation deliverables. External account credentials and final business identity cannot be invented; section 34 assigns those launch inputs with safe development defaults. No ordinary workflow decision is intentionally left to the developer to guess.

02 Release decisions and boundaries

ID	Decision	Release baseline
DEC01	Platform	iPhone first; iOS 17 or later as the product minimum, raised only if selected supported dependencies require it and documented before sprint one. Current Apple submission SDK requirements override this minimum.
DEC02	Customer interface	Responsive browser portal, Safari and Chrome, no customer app or persistent customer account.
DEC03	Geography and language	US storefront, English US, USD only; US business address. Customer browsers may access from other locations.
DEC04	Tenant model	One verified owner and one business workspace per account. Same owner can use two signed-in iPhones; no staff invitations or ownership transfer in v1.
DEC05	Payments	Record payments and refunds received outside the app. No card entry, payment collection, escrow, financing or payout promises.
DEC06	App monetization	Three lifetime free published jobs, optional 14-day app-managed trial, monthly and annual Apple subscriptions.
DEC07	Approval	Email code verification followed by explicit approval of an immutable document revision. No claim of notarization, identity certification or universal legal enforceability.

ID	Decision	Release baseline
DEC08	Documents	One approved base quote, sequential approved extra-work orders, one active final invoice per job; revisions, voiding, limited invoice credits and manual payment corrections are defined below.
DEC09	Tax	Operator-selected tax-exclusive rate per line. No jurisdiction inference, automated tax advice, filings or claim of nationwide tax compliance.
DEC10	Connectivity	Offline draft editing and cached reading. Sending, approval, financial ledger changes, invoice issue and purchase verification require connectivity.
DEC11	AI	No custom AI model, autonomous pricing, image interpretation or generative contract terms. Ordinary keyboard dictation may be used.
DEC12	Distribution	No tracking SDK or advertising identifier in v1. First-party aggregate funnel measurement; no fingerprinting or cross-app tracking.
DEC13	Support	In-app support and a monitored public support address. No 24-hour support claim unless staffed.
DEC14	Scope reductions	Before invoicing, use a customer-approved reduction order. After invoicing, use an owner-issued credit note with reason. Neither edits prior approvals.

Included in v1: authentication, business setup, customer records, service-item catalogue, draft recovery, quotes, additions and reductions, secure approvals, invoice creation, PDF output, manual payments/refunds, credit notes, subscription management, exports, account deletion, notifications, limited support console, first-party analytics and operational monitoring.

Deferred: Android operator app, full operator web app, staff seats, customer accounts, automated payment reminders, payment-provider links, recurring invoices, deposits invoiced separately, progress billing, multiple invoices per job, card processing, accounting integrations, inventory, scheduling, dispatch, GPS, mileage, payroll, expense OCR, pricing AI, voice-agent interaction, multi-currency, inclusive VAT and tax filing. Android is a separate release with its own billing and policy acceptance gate; shared TypeScript does not make it automatically complete.

Manual payment entries may be recorded only after an invoice exists. A pre-invoice deposit can be represented as an informational note and recorded with its actual received date when the final invoice is issued. The app must make clear that a note does not reduce the balance. Users requiring deposit invoices or progress claims are outside the first release audience.

03 Product outcomes and validation

ID	Metric	Exact definition and initial target
MET01	Activation	Verified owner publishes a real non-demo quote or direct invoice within seven days of signup. Target at least 35 percent of qualified signups; hypothesis.
MET02	Core workflow adoption	Among owners with at least one approved quote, percentage with a customer-approved change in 30 days. Segment by owners reporting extra work; do not force changes to inflate it.

ID	Metric	Exact definition and initial target
MET03	Repeat job use	Activated owner publishes another distinct real job within 30 days. Initial pilot gate at least 50 percent of operators who had another eligible job.
MET04	First invoice speed	Median active editing time from invoice preview open to issued invoice for an approved job under two minutes; exclude idle intervals above 30 seconds.
MET05	Reliability	Zero known lost committed documents, duplicate financial events or cross-tenant disclosures at release. Monthly crash-free sessions target at least 99.5 percent.
MET06	Monetization	Paying owner is server-verified active subscription with first transaction not refunded; measure conversion after full trial observation, not at trial start.
MET07	Acquisition	CAC equals attributable media cost divided by non-refunded new paying owners; report unknown attribution separately. Initial target USD 40 to 50 is a business hypothesis.
MET08	Retention	Monthly subscription renewal by acquisition cohort and weekly business activity. Annual subscription purchase is not evidence of annual retention.

Interview at least 20 eligible operators before public launch. Ask for actual recent examples rather than hypothetical preference. Five paid pilot commitments and repeated usage are the initial business gates. These are capital-allocation criteria; they must never alter accounting calculations or app authorization.

04 Roles and permissions

Actor	Allowed	Explicitly prohibited
Business owner	Manage own workspace, customers, drafts, send requests, issue invoices, record settlements, manage own subscription, export and delete account	Approve on a customer's behalf, modify issued snapshots, change another tenant, grant own trial resets
Customer recipient	View a scoped document after email verification, approve or decline the current quote/change, obtain own document and approval receipt	Browse workspace, edit prices, approve superseded revision, see internal notes or other customers
Support agent	View limited operational metadata, inspect delivery status, resend transactional messages through approved action, create support ticket	Read job content or photos by default, change invoice amounts, approve documents, edit payments, reveal tokens
Support supervisor	Time-limited explicitly granted access to a specified support case, revoke compromised links, suspend abusive outbound sending, audited billing investigation	Silent impersonation, unrestricted exports, automatic changes to customer commercial records

Actor	Allowed	Explicitly prohibited
Worker	Run a specific queued task using scoped identifiers	Trust unvalidated payload tenant IDs or execute unbounded user-provided URLs/code
Infrastructure operator	Maintain deployment, recover backup under logged incident procedure	Routine browsing of customer content or bypassing application audit for convenience

AUTHZ01: Every resource read and mutation derives workspace membership from verified server identity. A workspace ID in a route or body is never evidence of authorization. Cross-tenant object references return generic 404. Public approval sessions are scoped to one request and cannot exchange into owner sessions. Support permissions use separate staff identities with mandatory MFA and no shared accounts.

05 Domain vocabulary and invariants

A job groups one customer and one site. A quote is the proposed initial scope. A document revision is the immutable rendered-and-hashed snapshot published to a customer. A change order modifies the accepted scope before invoicing. A source line is an approved line from the base quote or an addition. A reduction consumes part of the remaining net/tax value of a source line. An invoice is a fixed billing snapshot. A credit note reduces an issued invoice without overwriting it. A payment/refund entry records money handled elsewhere. An approval request controls verified customer access and the decision window.

INV01: Monetary amounts use integer USD cents; quantity uses a decimal string with at most three decimal places. Never calculate money with binary floating point. Server results are authoritative; mobile uses the same shared calculation package for previews.

INV02: Accepted quotes, accepted changes, issued invoices, issued credits, decisions and ledger entries are immutable. Corrections create a new version, reversal or credit; no UPDATE may rewrite an accepted commercial fact.

INV03: A document revision includes a canonical snapshot and SHA-256 digest of those canonical bytes. PDF bytes have a separate digest. A digest is an integrity control, not evidence that a person was legally identified. JSON key ordering, number/string representation and array order are fixed by schema version.

INV04: At most one pending approval request exists for a job. Each request binds document revision, recipient email, scope version, expiry and consent-text version. Approval and cancellation race on one database lock; exactly one terminal result wins.

INV05: Approved additions/reductions update the job scope version atomically. Final invoice issue locks that scope version and rejects pending requests or unpublished change drafts until explicitly resolved. No approved charge disappears because the user edited a catalogue item or business profile.

INV06: Financial numbering is unique and monotonic within workspace and document type. Numbers are never reused after voiding. An allocated number lost to a failed issue is retained as a void/reserved audit record rather than silently recycled.

INV07: Payment status is derived from invoice, credits, posted payments and refunds. “Mark paid” is an entry flow, not a writable status flag. Owner-reported payment does not mean independently verified bank settlement.

INV08: Archived data is hidden from default lists, not deleted. Subscription expiry never removes record access or completion rights on jobs already published. Account deletion is a distinct intentional workflow.

INV09: A published job counts once against the lifetime free limit or trial publication cap. Revisions, changes, invoices, credits and retries do not consume additional jobs. Deletion, archiving, cancellations and reinstalls do not replenish used slots.

INV10: Server UTC is authoritative for acceptance and expiry. Local dates and times are displayed using the workspace IANA timezone; date-only fields never round-trip through midnight UTC conversion.

06 End to end journeys

First useful job

JRN01: New operator sees the product promise and a sample document, then selects Create my first quote. The operator enters email and a code, completes three short setup steps, adds customer and site, adds line items, and previews totals. Publishing shows a review sheet with recipient and expiry. Confirming publishes a frozen revision, consumes one job slot if necessary, and queues delivery. The operator sees Published and a separate delivery indicator. A queued email is not labelled Delivered.

Customer approval

JRN02: Recipient opens the secure link. The landing page reveals only business display name and document type. The recipient requests a code sent to the bound email, enters it, and reviews scope, prices, tax, total, expiry, terms and attachments. The recipient enters their name, checks an unticked acknowledgement and selects Approve quote or Approve extra work. The server records the exact revision and returns a durable receipt. A refresh shows the same decision and cannot create another approval.

Additional work

JRN03: For an accepted quote, the owner chooses Add extra work, optionally takes a photo, enters description and price, and sees the change total and new job total separately. The recipient approves the current change. It joins the accepted scope. A later change starts from the new scope version. The original approved quote remains unchanged.

Reduced scope

JRN04: Owner selects Reduce agreed work and chooses an approved source line with remaining value. Owner specifies a net reduction in dollars and a reason. Tax reduction is calculated automatically from the original tax amount. Recipient approves the reduction and sees old job total, reduction including tax, and new job total. The system cannot reduce a source line below zero or the job total below zero.

Invoice and money tracking

JRN05: Owner chooses Create final invoice. The app resolves every change draft or pending request first, then previews accepted scope, additions, reductions, invoice due date and payment instructions. Confirm issues the invoice. Owner may share it, record money received, issue a correcting credit, or record a refund. The customer sees outstanding balance labelled as reported by the business.

Direct invoice without quote

JRN06: An operator billing completed work may choose New job then Invoice without quote. The owner enters line items and issues an invoice without a customer approval flow. The document explicitly states that it was not preceded by in-app scope approval. This job cannot later add pre-invoice changes. This exception delivers ordinary invoicing without implying customer consent.

Interrupted work

JRN07: Owner loses connectivity during draft editing. Saved on this device is shown after local persistence, not before. Publish is disabled with a reconnect explanation. On recovery the draft synchronizes using version checking. A conflicting remote edit prompts a comparison and deliberate choice; the app never silently overwrites either version.

07 Navigation and visual specification

The owner app has four bottom tabs: Jobs, Customers, Items and Settings. Jobs is the default. A prominent New job action remains accessible on the Jobs screen; avoid a floating control that obscures list content. Job detail is the central workspace with Overview, Documents and Activity sections in one scrollable page. No dashboard of speculative revenue or “money recovered” claims.

UI01: Use system San Francisco typography; body 17 pt, secondary 15 pt, section title 22 pt semibold, screen title 28 pt bold. Support Dynamic Type through accessibility sizes. Use an 8 pt spacing scale, 16 pt screen gutters and 12 pt corner radius. Primary navy #17324D, background #F7F8FA, text #17212B, secondary #52606D, border #D5DCE3. Status colours require text and icons, never colour alone. Test every actual text/background pair for accessibility.

UI02: Buttons minimum 48 pt height and 44 by 44 pt tappable region. Primary action is full-width at the end of forms; long forms may use a safe-area sticky action bar. Keep destructive actions separated and labelled with consequence. Numeric inputs use decimal keyboards; prices display USD prefix; quantity has a unit label. Respect device safe areas, keyboard avoidance and screen-reader focus.

UI03: V1 light appearance is the designed baseline; explicitly opt out of unsupported automatic dark inversion. Web portal uses a single column up to 720 CSS px, 16 px mobile gutter, 18 px body text, 48 px buttons. Customer line items stack label and quantity/price/tax on narrow screens. All customer actions remain reachable at 320 CSS px width and 200 percent zoom.

UI04: Every data screen has loading, empty, loaded, refresh failure, offline and access-expired behaviours. Cached content stays visible during refresh with a timestamp. Skeleton loading must not shift controls dangerously. Inline validation preserves all entered values and focuses the first invalid field. Error banners have Retry only when retry is safe.

08 Screen inventory and field contracts

Each screen ID is a required design frame and QA coverage unit. Screen layouts are specified here; the implementation designer must deliver matching Figma or equivalent editable layouts, interactive core-flow prototype, and all state variants before UI signoff. These design deliverables are part of the development contract, not claimed attachments to this PRD.

Screen	Structure and primary action	Required states and navigation
S01 Welcome	Promise, sample document, Create my first quote, Sign in, terms/privacy links	No purchase prompt; demo uses fictional data and cannot send
S02 Email sign in	Email input, Send code; explain passwordless login	Generic code-sent response, throttled, network failure
S03 Verify code	Code field, masked email, resend countdown, Change email	Expired/incorrect code, attempt limit; allow OS paste/autofill

Screen	Structure and primary action	Required states and navigation
S04 Business setup	Three steps: name/trade, business contact/address, timezone and document defaults	Back preserves data; optional logo can be skipped
S05 Jobs	Search, Active/Finished/Archived filters, job cards, New job	Empty state and draft-sync badges; cursor pagination
S06 Create job	Customer, site address, title, Quote or Direct invoice	Customer creation sheet; no forced contacts permission
S07 Customer form	Name, email, optional phone, billing address	Duplicate warning, archived customer restore option
S08 Job overview	Customer/site, scope total, current step, document list, activity	Next action varies by state; no generic editable status
S09 Quote editor	Ordered line cards, Add item, notes, terms, expiry	Draft save indicators, conflict recovery, validation
S10 Line editor	Description, quantity, unit, unit price, fixed discount, tax rate	Live subtotal and net/tax; saved-item picker optional
S11 Preview and publish	Frozen-looking preview, recipient, expiry, publish confirmation	Show slot/paywall condition before submission; stale-version recovery
S12 Request detail	Status, revision, sent-to email, delivery and decision events	Resend, withdraw, replace recipient as permitted
S13 Extra work editor	Reason, added lines, photo attachments, old/change/new total	Only after quote acceptance and before invoice
S14 Reduction editor	Eligible source line, reduction amount, reason, tax/new total	Bound max value; block all-zero change
S15 Invoice preview	Source summary, line details, due date, instructions, issue action	Block unresolved changes; explicit direct-invoice label
S16 Invoice detail	PDF/share, issued total, credits, received/refunded/balance, ledger	Record payment, Credit, Record refund, Void if permitted
S17 Payment or refund	Amount, date, method, optional reference, confirmation	Overpayment warning; refund maximum; online only
S18 Credit note	Select invoice lines, net credit amounts, reason, preview/issue	Tax calculated; cumulative caps; irreversible issue warning
S19 Customers	Search/list, create, customer detail with jobs	Archive not destructive delete where referenced
S20 Items	Search/list, default price/unit/tax, add/edit/archive	Changes never alter existing documents
S21 Subscription	Free usage/trial date, monthly/annual prices, buy/restore/manage	Pending, active, canceled-but-active, expired, refund, sync failure
S22 Settings	Business defaults, notifications, support, privacy/export/deletion, sign out	Sensitive actions reauthenticate; logo/address updates future only
S23 Conflict recovery	Local and server values, time, Keep server or Save local as draft copy	No automatic replacement of sent documents
S24 Export and deletion	Export status, download, deletion consequences and confirmation	Reauthentication, deletion progress, retained-record explanation

Screen	Structure and primary action	Required states and navigation
S25 Customer access	Business/type, masked email, request/enter code	Invalid/expired/revoked link; no private scope before verification
S26 Customer review	Full snapshot, price summary, attachments, consent/name, approve/decline	Loading, read-only decided, superseded, expired, revoked
S27 Customer receipt	Decision/time/revision, downloadable receipt and document, business contact	Approval does not claim payment; invoice portal read-only
S28 Support console	Case lookup, bounded metadata, allowed operational actions	Staff MFA, reason required, access expires, immutable audit

Field validation

VAL01: Business display name 2 to 100 characters; legal name 2 to 150; operator name 2 to 100; job title 1 to 120; customer display name 1 to 120. Trim outer whitespace, reject control characters except newlines in multiline fields, preserve Unicode names. Email up to 254 characters, normalized for lookup without provider-specific dot/plus rewriting; preserve original presentation. Customer email is mandatory for approval requests, optional for manually shared direct invoices. Phone optional, E.164 where parsable, otherwise show validation rather than guessing country.

VAL02: US address: line 1 max 150, line 2 optional max 150, city max 80, state two-letter selection, ZIP five digits or ZIP plus four. Site address is optional for remote/service invoices, but ask the user to explicitly choose No site address. Billing and site addresses are distinct. Business timezone defaults to the device IANA zone and requires confirmation; dates use MM/DD/YYYY presentation.

VAL03: Line description 1 to 500 characters; unit enum item, hour, day, square_foot, linear_foot or custom label up to 20; quantity greater than 0 and at most 999999.999; unit price 0 to 99999999 cents. Fixed line discount at most the rounded gross line value. At most 100 lines per document, total absolute amount at most 999999999 cents. Currency is USD and cannot be edited. Free lines are permitted but a quote must have at least one positive net line; invoices with zero balance after approved reductions remain valid.

VAL04: Public scope notes 0 to 2000 characters; commercial terms 0 to 4000; internal notes 0 to 4000 and never exported in customer PDFs. A reason for reduction, credit or void is required, 5 to 500 characters. Approval typed name 2 to 100 characters. Terms and notes are plain text with escaped display; no user HTML. All constraints run client-side for convenience and server-side for enforcement.

09 Accounts customers and catalogue

ACC01: Use email one-time codes for owner authentication through Supabase Auth. Configure six-digit codes, ten-minute expiry, 60-second resend cooldown and at most five verification failures per challenge. Account creation and sign-in share one flow; responses do not reveal existing accounts. Configure equivalent or stricter provider limits and verify the actual SDK behaviour in staging. Never build a second plaintext code store for owner authentication. Tokens use the supported secure-session mechanism; refresh tokens belong in OS secure storage, not application logs or analytics.

ACC02: Access tokens are verified server-side for signature, configured issuer/audience, expiry and account status using supported JWT verification. Expired access attempts trigger one refresh, then sign-in if unsuccessful. An offline owner may read cached records and edit drafts for up to seven days since last successful authentication; publishing still requires server authorization. Sign-out warns about unsynced drafts and offers synchronize or

discard local data. Device session revocation clears access at next request; backend account suspension is checked on every mutation.

ACC02A: For recent-auth operations, request a fresh owner OTP through the same provider, then obtain a backend single-purpose action grant with a five-minute expiry after verifying the provider authentication time. Store and consume its hash server-side, bind it to user/action, and send it in X-Action-Grant. The API inventory includes POST /account/action-grants {action} for export, deletion, email_change and replace_link; it requires a freshly authenticated session. Ordinary token refresh alone is not fresh authentication. Add action_grants(id,user_id,action,token_hash,expires_at,used_at) to the restricted identity schema.

ACC03: Email change requires recent reauthentication, verification of the new email, and notification to the old email. Changing owner email does not change the stable user UUID, workspace, customer recipients or billing association. A lost owner email requires a documented identity-recovery support process; staff cannot bypass it by matching an invoice number. Account ownership transfer is unavailable in v1.

CUS01: Customer records can have identical names. Warn on duplicate normalized email within a workspace but permit separate named contacts after confirmation. A job chooses one approval contact. Editing customer details affects future drafts only; published snapshots retain original details. Before publication explicitly select Apply current contact details to draft if the draft contains an older snapshot.

CUS02: Archive a customer to remove it from new-job pickers. Existing jobs and documents remain accessible. Deleting an unreferenced customer is allowed; if referenced, offer archive and export instead. A privacy request concerning a customer is handled through the privacy process because approved commercial records may contain historical references. Do not automatically rewrite historical records on a contact deletion request.

CAT01: Saved items hold description, unit, suggested quantity, unit price cents, fixed discount default zero and tax rate default zero. Defaults are copied into a new line, never linked live. Seed five editable zero-price examples such as Labour hour, Materials, Small repair, Installation and Disposal; explain that users set their own prices. No invented industry rates. Archive retains historical uses. Catalogue search matches description case-insensitively; a blank query lists most recently used items.

10 Quote and job lifecycle

Job state rules

JOB01: Persist lifecycle as draft, active, invoiced, finished, canceled or archived, plus archived_from_state. Draft becomes active on first publication. Accepted quote does not mean work completed. Invoiced means a final invoice has been issued. Finished is an owner action allowed after an invoice is settled or explicitly waived by credits; finishing never deletes records. A canceled job can retain a receivable and must visibly show it. Archive is a visibility action allowed from any non-draft state only when no approval is pending; it does not alter financial status.

JOB02: A draft job can be deleted when it contains no published document. Published jobs cannot be deleted individually in v1. Cancel requires a reason, withdraws pending approvals and blocks new work; an existing invoice remains collectible until credited or voided. An active accepted job with no invoice can be canceled; notify the customer and preserve the prior acceptance. This is an operational cancellation notice, not a legal determination that all contractual obligations ended. Reopening a canceled job is unavailable; create a linked new job.

Quote document states

From	Command	To and effects
draft	Save	draft with incremented draft version

From	Command	To and effects
draft	Publish	issued snapshot and pending request; job becomes active
pending request	Customer approve	accepted snapshot; job scope version increments
pending request	Customer decline	declined request and snapshot retained
pending request	Owner withdraw	withdrawn request; snapshot remains readable as withdrawn
pending request	Publish replacement	Prior request superseded and new snapshot/request created atomically
pending request	Time reaches expiry	expired; approval endpoint rejects even if worker has not updated display
declined or expired or withdrawn	Create revision	New editable draft copied from old snapshot; new approval required
accepted	Edit request	Rejected; use a change order or cancel job

QUO01: Quote expiry defaults to 14 calendar days from publication, interpreted as 23:59:59 in workspace timezone on the selected date. Owner may choose 1 to 90 days. The server stores resolved expires_at UTC and the original local date/timezone. Do not adjust expiry after send; extending a quote creates a new request and frozen revision requiring approval. A request can be resent without extending expiry.

QUO02: Quote labels use Q-000001 per workspace. Drafts display Draft and a local short identifier, not an official number. Revisions share the original quote number with revision suffix R1, R2 and so on. First publication is R1. Number allocation, snapshot insert, slot consumption, pending request and outbox event commit in one transaction. PDF/email generation happens after the commit.

QUO02A: The preview response includes snapshot schema version, frozen business/customer defaults and a ten-minute preview validity. Publishing after that interval returns PREVIEW_CHANGED and requires regeneration. Canonical bytes use UTF-8, sorted object keys recursively, explicit nulls for nullable snapshot fields, decimal quantities as three-place strings, integer cents and stable position/UUID-sorted arrays; exclude request tokens, live delivery status and generation timestamps. Store the canonical byte representation alongside the parsed snapshot so later runtime differences do not alter its digest.

QUO03: Changing recipient on a pending request requires withdrawal and a replacement request/revision, even if pricing is unchanged. The old recipient cannot approve afterward. An accepted quote cannot have its recorded signer replaced. A new approval contact for future changes requires owner confirmation and is independently verified for that new request.

11 Change orders and reductions

CHG01: A change order requires an accepted base quote and no issued invoice. Owner can create multiple local drafts but only one published pending order. Before publishing, rebase its preview onto the latest accepted scope version and require owner review when that version changed. Invoice issue blocks while any change draft exists until the owner chooses Publish or Discard; discarded drafts are explicitly confirmed.

CHG02: Addition orders contain one or more normal positive lines. Reduction orders reference one or more approved source lines and specify positive net-credit amounts in cents; no negative unit prices or quantities. A single change order may contain additions and reductions when replacing work; customer sees each separately and the net change. Require a nonzero net or a meaningful scope replacement with a written reason; a zero-value replacement still requires approval if scope changed.

CHG03: A source line retains original net cents and tax cents, less all previously accepted reductions. A draft reduction can reference a base-quote or previously approved addition line, not another reduction, an unapproved line or an unrelated job. Cumulative net reductions cannot exceed the source net. Exactly one pending request prevents simultaneous competing commitments; approval still rechecks caps in the transaction.

CHG04: Change labels use CO-000001 per workspace, revision suffixes as for quotes. Customer presentation shows “Previously agreed total”, additions, reductions, “Change including tax” and “New agreed total”. Financial totals and source references are immutable once sent. Approval increments scope version and marks the included source values consumed atomically. Declining, expiring or withdrawing never applies the change.

CHG05: Accepted changes cannot be deleted or silently rolled back. To reverse an accepted addition, issue a new reduction referencing its remaining source value. To restore a reduction, add a new positive line with a reason and explicit new approval. Never fabricate an approval to repair data. Subscriptions do not block changes on grandfathered published jobs.

12 Approval security and evidence

APR01: Generate a 256-bit random request token; the request table stores its keyed hash, key version and request ID. The raw value exists only in process memory and an encrypted, access-restricted delivery payload until submission completes; purge that payload within 24 hours of terminal delivery status. Never store raw tokens in ordinary outbox JSON. Use a URL fragment for the raw token, such as /review#token, so it does not enter ordinary server request logs. The portal exchanges it via POST, clears the fragment with history replacement, and holds a Secure HttpOnly SameSite cookie. Native browser fallback must not leak the token in a referrer, error report, email analytics or link preview. All portal responses use no-store and no-referrer controls.

APR01A: Token exchange creates a 20-minute preverified cookie session and returns a random CSRF nonce bound to that session. Hold the nonce in page memory and return it in X-CSRF-Token for portal mutations; rotate it after successful code verification. Validate Origin against the portal origin. Owner API uses bearer tokens and does not accept portal cookies as owner authentication.

APR02: Before verification, reveal only business display name, document type, masked destination email and access state. Request a six-digit code sent to the bound recipient. Hash public codes with a server-held secret; expiry ten minutes, one active challenge per request, five failed attempts, resend no faster than 60 seconds, maximum five sends per hour per request and 20 per IP with an adaptive abuse limit. New code invalidates the old. Provider delivery failures do not count as verified identity. No code is returned in an API response.

APR03: Successful code verification grants a one-hour scoped customer session. Viewing a decided document is permitted through a newly verified session while its link remains enabled. Approval request token access lasts 90 days after terminal decision, then requires owner-issued fresh read-only link. If the original PDF is still preparing, the page shows a retryable preparation state and disables approval until ready. A pending approval expires according to its commercial expiry even if token access continues. An explicit owner resend or replace-link command rotates the token and invalidates old sessions while preserving request identity, document version and commercial expiry. Warn that older links will stop working. A worker retry of the same delivery attempt reuses its encrypted payload and does not rotate again.

APR04: Approval screen requires the original reviewed PDF artifact in ready state as well as full accessible snapshot text, typed name and explicit unticked acknowledgement: “I have reviewed this version of the scope and price and approve it. I understand this records my approval electronically.” Display the commercial terms separately. Do not require agreement to marketing, app subscription or unrelated privacy processing. No legal-grade signature certificate claim. If enforceability is required for a specific trade/state, the business owner must obtain reviewed wording and release approval before making that claim.

APR05: Store request ID, document revision ID, snapshot digest, typed name, verified email, decision enum, server timestamp, consent text version and consent text snapshot. Store a minimized user-agent family and IP evidence encrypted with restricted access, with retention described in section 25. Do not collect GPS, contact lists or biometrics. The evidence receipt lists which verification was performed and never asserts government identity verification.

APR06: Approve requires the request still pending, unexpired, not revoked, exact expected scope version and exact snapshot hash. Lock request and job, check all predicates, insert one decision, update status/scope and outbox in the same transaction. Same idempotency key returns same receipt. A different approval after terminal decision returns existing decision with ALREADY_DECIDED, never another charge. Concurrent decline/approve: first committed valid command wins, later command receives 409.

APR07: Decline requires an optional comment up to 1000 characters. Store it privately for the owner and in the decision receipt, escaped. After decline, offer Contact business and Download reviewed document. Editing prices or renegotiating in the portal is unavailable. No chat feature.

APR08: Owner receives in-app status and transactional email after approval/decline. Customer receives the reviewed PDF and receipt link. A failed receipt email does not undo the committed approval; it appears as Delivery failed with retry in owner view. Download remains available to the verified recipient.

13 Monetary calculations and fixtures

FIN01: Quantity q is decimal to three places; price p is integer cents. Round q times p half-up to whole cents to obtain gross. Subtract fixed per-line discount d to obtain net. Rate r is integer basis points from 0 to 2500, representing 0 to 25.00 percent. Tax equals half-up of net times r divided by 10000. Sum line net and line tax; document total equals those two sums. No document-level discount, compounded tax, tax-inclusive prices, withholding, foreign currency or currency conversion in v1. The 25 percent UI cap is a product constraint, not a statement of law.

FIN02: Rates and applicability are entered by the operator. Defaults are zero with “Confirm tax treatment for your business” setup guidance, not an assertion that zero is correct. Public documents show the rate and tax amount for each taxable line and a summary grouped by rate. Values that cannot be represented by this simple model are outside v1; the app must not advertise comprehensive US tax compliance.

FIN03: For a source line with original net N and tax T , let prior cumulative accepted net reduction be C and a new reduction be x . New tax reduction is $\text{round_half_up}(T \text{ times } (C+x) \text{ divided by } N) \text{ minus } \text{round_half_up}(T \text{ times } C \text{ divided by } N)$. Cap $C+x$ at N . This cumulative method ensures all tax is reversed when the entire source net is removed and prevents penny drift across partial reductions. N cannot be zero for a reducible line.

FIN04: Invoice lines represent approved additions and base lines with net/tax after accepted reductions, retaining source IDs and reduction history in the appendix. Snapshot precomputed net and tax; do not recalculate from current tax defaults. Invoice credits use the same cumulative method against these residual invoice line amounts. An invoice credit is a billing correction, not an edit to the accepted job scope. Summaries explicitly separate Agreed job total, Invoice issued total and Credits.

FIN05: Posted payment amounts minus posted refunds produce net money received. Reversals cancel specific erroneous entries and are not additional cash movements. Balance equals issued invoice total minus issued credits minus effective payments plus effective refunds. Positive means Amount due; zero means Settled; negative means Amount to refund or reconcile. Never hide an overpayment with $\max(\text{balance}, 0)$ in the ledger.

Fixture	Inputs	Expected result
---------	--------	-----------------

Fixture	Inputs	Expected result
F01 Basic tax	Quantity 2.5, unit price 10000 cents, discount 1000, rate 825 bp	Gross 25000; net 24000; tax 1980; total 25980 cents
F02 Quantity rounding	Quantity 0.333, unit price 1000 cents, no tax	Gross and net 333 cents
F03 Half cent tax	Net 5 cents, rate 1000 bp	Tax 1 cent; total 6 cents
F04 Approved addition	F01 base plus added line net 10000 at 825 bp	Change total 10825; new scope 36805 cents
F05 Sequential reductions	Source N 100, T 8; reduce 33 then 33 then 34 cents	Tax reductions 3 then 2 then 3; source remaining zero
F06 Over reduction	Source remaining net 34; request reduction 35	Reject 422 CREDIT_EXCEEDS_SOURCE; no state change
F07 Partial payment	Invoice total 10000, payment 4000	Amount due 6000; partially_paid
F08 Credit after payment	Invoice 10000, paid 10000, issued credit 2000	Balance -2000; refund_due
F09 Refund resolves credit	F08 plus posted refund 2000	Balance zero; settled
F10 Overpayment	Invoice 10000, payment 12000 confirmed by owner	Balance -2000; refund_due; never paid-only display
F11 Reversal	Invoice 10000, erroneous payment 4000 then its reversal	Balance 10000; no duplicate refund
F12 Full scope reduction	Accepted base fully reduced before invoice	Final invoice may total zero; settled without fake payment

The shared money module must ship property tests for nonnegative remaining source amounts, full-credit tax reversal, associativity of document summation, repeated retry invariance and equality between mobile preview/server/PDF. No locale-formatted string is accepted as a stored number. API cents fit within the documented maximum and serialize as integers; larger future totals require a schema revision.

14 Invoices credits payments and corrections

BIL01: Invoice issue requires an accepted base quote or an explicitly direct-invoice job. For quote-based jobs, every included source is accepted and no unresolved change draft/request remains. Owner reviews due date, customer billing snapshot and instructions. Default payment term is 14 calendar days, with options Due on receipt, 7, 14, 30 or a chosen date up to 365 days. Due date cannot precede issue date. Issue date is server date in workspace timezone; payment received dates may be earlier for a deposit recorded after issue.

BILO2: Allocate INV-000001 and freeze the invoice in one transaction. Job scope is then closed to change orders. A worker builds a PDF from the frozen snapshot, uploads it privately and sends when ready. UI distinguishes Issued, PDF preparing, Ready to share and delivery state. A PDF failure leaves an issued invoice and retry task, not an editable financial draft. Duplicate issue commands return the original invoice. No second active final invoice for the same job.

BILO3: An unpaid issued invoice can be voided only when no effective payment, refund or issued credit exists. Void requires reason, revokes active invoice links, notifies prior recipient and retains the number. A replacement invoice references the voided one and is numbered anew. Customer/typographical billing details may be corrected in replacement; approved scope amounts cannot be changed without a separate credited/rebuilt job workflow. Reissue from the same accepted scope is allowed; scope re-negotiation after issue is outside v1 and uses a new linked job.

BILO4: Credit notes CN-000001 are immutable, owner-issued reductions of selected invoice line net amounts with automatically allocated original tax. Require reason and preview. Cumulative credited net cannot exceed residual invoice line net. Credits cannot increase an invoice or record money returned. An erroneous issued credit cannot be deleted in v1: explain the limitation before confirmation and direct the owner to create a linked new invoice/job for any additional charge, with no automatic customer approval claim. This deliberately avoids a complete accounting journal system.

BILO5: Record payment accepts positive cents, received date no later than today in workspace timezone and no more than five years earlier, method enum cash, check, bank_transfer, external_card, other, reference up to 100 and note up to 500. Overpayments require an explicit confirmation showing the resulting refund balance. "Mark paid" opens this form with current amount due; it does not bypass confirmation. All records say Recorded by business; no bank verification badge.

BILO6: Record refund is enabled only when effective paid money exists and balance is negative. Amount may not exceed either outstanding refund balance or remaining effective payments. It represents money already returned externally; show "This records a refund you have already made. It does not send money." Date and method follow payment rules. A refund can be partial. Allocate refund cents server-side against effective payments oldest first in ledger_refund_allocations; cumulative effective allocation cannot exceed a payment. Reversing an erroneous refund releases its effective allocation without deleting history. Zero entries are prohibited.

BILO7: An erroneous payment or refund can be reversed with mandatory reason, referencing exactly one original entry. It can be reversed only once; a reversal itself cannot be reversed in v1. To correct amount/date/method, reverse then create a new entry; retain both visibly. Reversing a payment with dependent refunds is blocked until erroneous dependent refund records are reversed or the owner resolves the ledger externally. Reversal transactions must re-evaluate invariant checks under invoice lock. An actual refund must never be reversed merely to cosmetically permit invoice voiding.

BILO8: Derived invoice states are issued_unpaid, partially_paid, settled, overdue or refund_due, with separate voided flag. Overdue applies when balance is positive and due date precedes the workspace current date; no late fees accrue. Credit-to-zero is Settled by credit, not Paid. A receipt for a manual payment is optional owner sharing with explicit recorded wording; automatic bank/payment confirmations do not exist.

15 Subscription and entitlement contract

SUB01: V1 has entitlement code pro. Products are provisional identifiers jobtoinvoice.pro.monthly and jobtoinvoice.pro.annual in one Apple subscription group, nominal US prices USD 19.99 and 149.99. Final bundle prefix and store product identifiers must be recorded before release. Always display StoreKit-localized price and billing period, not hardcoded price text. The annual option states total annual charge plus monthly equivalent as secondary information. Monthly is selected by default. No weekly plan or lifetime offer.

SUB02: The free tier permits three distinct published jobs over the account lifetime. Unlimited local drafting within abuse limits is permitted, but publication of a fourth new job requires trial eligibility or active Pro. A job slot is consumed atomically on first quote publication or direct invoice issue and never again. A slot reservation rolls back if publication fails before commit. A document delivery failure after commit does not restore the slot. Trial/paid jobs are marked with publication entitlement so they do not consume unused lifetime free slots.

SUB03: The optional 14-day trial is app-managed, not an Apple introductory subscription offer. It starts only when the verified owner explicitly taps Start 14 day trial; no card, no automatic charge, no StoreKit purchase sheet. Store start/end in UTC server time once per owner/workspace, never reset by reinstall or account email change. Trial permits at most 20 new published jobs, disclosed as “Try all features on up to 20 jobs for 14 days.” Trial expiry and job cap both apply. At expiry, owner can use any unconsumed free slots or purchase Pro; trial jobs keep completion rights.

SUB04: Active paid Pro permits new jobs with a disclosed operational fair-use publication cap of 1000 per rolling 30 days and 10 GB retained attachment storage. This is not marketed as unlimited. Internal alerts at 80 percent usage and a support path precede the cap. Quota restrictions apply only to new job publication/uploads, not record access, invoice completion, credits or payment/refund tracking. Trial storage cap 500 MB; free tier 250 MB. Existing uploaded attachments remain readable after downgrade; prevent additional uploads beyond cap but allow text-only completion.

SUB05: Every published job grants completion rights independent of subscription: edit permitted drafts belonging to that existing job, send new pre-invoice change requests, issue its invoice, record payments/refunds, issue credits, resend documents and export. New jobs and duplication into new jobs are subject to current entitlement. A job is a single site/scope and cannot be reset to a new unrelated customer after publication. This limits abuse without holding unfinished work hostage.

Billing condition	Entitlement and UI behaviour
Purchase pending or customer cancels sheet	No new Pro access; existing free/trial rights remain; never show purchase success
Verified active	Pro until verified entitlement expiry; show renewal period and manage link
Auto renewal canceled	Pro remains until paid-through expiry; label “Ends on date”
Store billing grace period	Honor verified provider entitlement while active; show billing issue and manage link
Billing retry with no active entitlement	New Pro publication blocked; completion/read rights remain
Refunded or revoked	Reconcile provider state, remove prospective Pro rights; do not delete customer records
Provider unavailable	Keep last server-verified access up to its known expiry; do not manufacture extensions
Expired	Fall back to unused free slots and completion rights; offer monthly/annual plans
Trial ended	No charge; clearly state end and remaining free slots

SUB06: Require login before purchase and initialize RevenueCat with stable owner UUID; do not purchase under anonymous identity. Keep receipt association with its existing app user on restore; automatic transfers to another owner/workspace are disabled. Restore purchases triggers server reconciliation and either restores the same-owner entitlement or displays “This purchase is linked to another account. Sign in to that account or contact support.” Do not expose the other account email. Test the configured RevenueCat transfer behaviour with two app accounts and one Apple account before release.

SUB07: Client purchase success is provisional until backend reconciliation; call the provider server API with secret credentials. During synchronization show “Purchase received. Checking access...” and preserve transaction context. Webhooks are triggers, not blindly trusted entitlement assignments. Authenticate webhook, deduplicate provider event ID, durably enqueue, query authoritative subscriber state and update snapshot. Out-of-order events cannot override a newer verified state. Poll/reconcile on app foreground when stale over 15 minutes and nightly for active paid accounts, with rate limits and batching.

SUB08: Do not implement custom proration or grant a second entitlement when switching monthly/annual. Initiate supported StoreKit plan change in the same subscription group and use the resulting provider effective dates. Explain price and effective timing using verified store state. Manage subscription opens the supported Apple management surface. Restore, terms, privacy and support are always visible on the paywall. The app-managed trial and paid purchase must never be described as the same offer. Sources SREF01, SREF03 and SREF04 inform implementation; verify SDK-specific behaviour in sandbox.

16 Draft persistence and synchronization

SYNC01: Mobile maintains a per-owner SQLite database with encrypted storage using the selected supported SQLite encryption build and OS-secured key, plus protected attachment files. Expo development builds are required; Expo Go is not a release environment. On first implementation spike, confirm SQLCipher support in the selected SDK/build and OS backup exclusion. If unavailable, choose a supported encrypted local persistence module before storing production records; plaintext fallback is forbidden. Uninstall or device loss can lose never-synced drafts, so the UI must distinguish local from cloud persistence.

SYNC02: Persist draft changes locally after a 500 ms debounce, and flush on blur/background as OS execution permits. Show Saving locally until transaction completes, then Saved on this device. While online, queue server draft writes within two seconds, then show Synced with timestamp only on acknowledgement. Show unsynced count in Jobs and Settings. OS background termination is not guaranteed to flush unfinished keystrokes; never advertise zero possible draft loss.

SYNC03: Each mutable resource has server version integer and updated_at. Every PATCH requires If-Match with that version. A local operation has UUID operation_id, target_id, base_version, mutation payload and dependency IDs. Server applies once and increments version. On 409 VERSION_CONFLICT, pause that resource queue and preserve both copies. Never use last-write-wins for amounts, recipients, terms or site identity. User can keep server copy or save local data as a new draft revision; publication always reviews current server version.

SYNC04: New local IDs are UUIDs accepted by server after owner/tenant validation. Sync dependencies in order: customer, job, document draft, line items and completed attachments. An attachment still uploading blocks publication if selected for the customer document; allow explicit Remove pending attachment to proceed. Local temporary drafts do not consume public document numbers or job slots.

SYNC05: Queued draft retries use exponential delay with jitter from 2 seconds to five minutes. Reconnect triggers a bounded queue drain; cap 20 concurrent draft operations per device and one per resource. Authoritative commands such as publish, approve, invoice issue, credits, payments/refunds, deletion and purchase reconciliation are never auto-issued from an offline draft queue. They require a live user action when connected. If a live command times out, query its operation result before offering another command.

SYNC06: Cache recent 90 days and up to 500 jobs, plus owner-pinned open jobs, subject to 250 MB document cache excluding pending attachments. Evict only server-confirmed cache using least-recently-used; never evict unsynced work automatically. Remote search requires connectivity; local search states “Searching downloaded jobs.” When switched to another account, lock and wipe previous cache after the unsynced-work confirmation. No sharing local database between accounts.

17 Documents photos and exports

DOC01: Use one versioned server-rendered HTML-to-PDF template for quotes, change orders, invoices, credit notes and approval receipts. US Letter, portrait, embedded licensed fonts, readable 10 to 11 pt minimum body, repeat item-table header, page X of Y, no clipped rows. Show business name/contact, document number/revision, issued date, customer/site, currency, lines, discounts, rate/tax, total, public notes and terms. No internal notes, authentication secrets, private customer data unrelated to this document or unverified approval badge.

DOC02: Quote/change PDFs show proposed or accepted/declined/withdrawn/superseded status on generated status copies, with snapshot content unchanged. Original reviewed bytes and digest remain preserved; an approval receipt references that original digest. A later status-stamped copy is a distinct artifact with its own hash, never overwrites original bytes. Invoice PDFs retain issue snapshot and provide a separate current balance/credit statement when requested; avoid silently regenerating the historical invoice as if it always had the later balance.

DOC03: Media allowed: JPEG, PNG and HEIC from camera or system photo picker, maximum 10 images per document and 10 MB per source image. No video, arbitrary files, GIF or SVG. Client resizes long edge to 2000 px and re-encodes; server verifies file signature, decodes, strips EXIF including location, re-encodes approved JPEG/PNG, and stores accepted output plus metadata. Reject malformed, oversized or decompression-bomb inputs. HEIC conversion must be proven on real supported devices. Original upload is purged within 24 hours after processing; no location retention.

DOC04: Photo permission is requested only when selecting Take photo. Photo picker avoids broad photo-library access. Permission denial offers Choose photo or Continue without photo. A photo attached to a published revision cannot be silently removed from the historical record. Owner chooses whether a draft photo is public or internal before publication; internal images never appear in recipient responses or PDFs.

DOC05: Object keys include random IDs and tenant namespace, never public customer names. Private buckets only. Signed download URLs expire after five minutes and are generated after authorization; browser responses do not cache private documents publicly. Signed upload authorizes one random object key and size/type constraints, expires after five minutes, and has no read permission. Completing upload is not equivalent to successful validation.

EXP01: Owners can export all workspace data from Settings after recent authentication, whether paid or expired. Asynchronous ZIP includes README, schema version, customers.csv, jobs.csv, document register, ledger.csv, immutable PDF documents, approval receipts and JSON snapshots; include accepted images with path mapping and exclude internal staff notes and secrets. UTF-8 CSV with formula-injection protection for cells beginning =, +, -, @ or control characters; numeric amounts have explicit cents columns and USD labels. Dates use ISO formats with documented timezone.

EXP02: Export snapshot has a consistent database cutoff; record cutoff timestamp. Target completion within 30 minutes for 1000 jobs/1 GB. Larger exports show progress and are split into numbered ZIPs with manifest and checksums. Download link requires owner authentication and lasts 24 hours; generated bundle deleted after seven days. No export email attachments containing full customer data. Repeated export request within 24 hours reuses same completed bundle or clearly requests a newer one; cap two new bulk exports/day/workspace.

18 Communications and exact transactional copy

NTF01: V1 delivery is transactional email, in-app activity and user-initiated OS share sheet. No automated SMS, WhatsApp sending, marketing campaigns or background calendar reminders. OS share-sheet completion does not prove the message was delivered. Owner may manually share the scoped link; approval still requires bound-email verification. Native push notifications are deferred. V1 owner notification preferences allow optional product/trial

reminders to be disabled; security, requested document delivery and commercial decision notices remain transactional. The app explains these categories and sends no marketing email without a separate opt-in.

NTF02: Email uses verified product sending domain, SPF, DKIM and DMARC. From display name is “[Business] via [Product]” with product-controlled envelope and authenticated sender; Reply-To is the verified business contact. Business-supplied email never becomes an unverified From header. Customer invoices without an approval flow use the same protected read-only recipient-access mechanism if sent by app email. A manual PDF share is explicitly owner-controlled disclosure.

Template	Subject	Required body and action
EMAIL01 Quote request	Review quote {number} from {business}	Business, quote total/currency, expiry, “Review quote”; explain email verification; no tracking pixel
EMAIL02 Change request	Review a change to your job with {business}	Old total, change including tax, new total, “Review change”; original approval unchanged
EMAIL03 Access code	Your {product} verification code	Code, ten-minute expiry, ignore if not requested; no commercial details
EMAIL04 Decision receipt	Your {decision} for {number}	Decision, revision, date/time/zone, verified email, download reviewed copy and receipt
EMAIL05 Owner decision	{number} was {approved or declined}	Customer display name, decision, open job; decline comment only within authenticated app
EMAIL06 Invoice	Invoice {number} from {business}	Issued total, due date, “View invoice”; states payment instructions come from business
EMAIL07 Credit	Credit note {number} from {business}	Credit total, referenced invoice, “View credit”; explicitly says this does not confirm a refund
EMAIL08 Withdraw or replace	Update to document {number}	Old request is no longer available for approval; new request link only when actually issued
EMAIL09 Trial ending	Your trial ends on {date}	Two days before end, “No automatic charge”; paid options and remaining free slots
EMAIL10 Export ready	Your business export is ready	Secure account link and expiry; no customer data in subject
EMAIL11 Deletion receipt	Your account deletion request	Effective lock time, removal timeframe, narrow retention explanation, support contact

NTF03: Outbox delivery states queued, submitting, accepted_by_provider, delivered, bounced, complained and failed. Accepted is not Delivered. Webhooks update state after signature verification/deduplication. Retry transient failures at 1, 5, 30 minutes and 2, 8 hours; five attempts then dead-letter plus owner notice. Permanent bounce suppresses address until owner corrects it through the revision/recipient process. A spam complaint suppresses automated sending to that address but leaves owner-controlled export available.

NTF04: One request may be manually resent no more than three times per day, at least ten minutes apart, with no automatic expiry extension. Record attempt ID and provider idempotency key. If provider submission times out after potential acceptance, reconcile before retry; do not blindly resend. In-app job activity is the authoritative delivery/status history, not the email inbox.

NTF05: Critical interface copy: offline publish “Reconnect to send this document. Your draft is saved on this device.” Conflict “This draft changed on another device. Review both versions before continuing.” Expired approval “This request has expired. Ask the business for a new version.” Stale approval “A newer version is available. This version cannot be approved.” Ledger warning “This records money handled outside the app. It does

not move money.” Subscription expiry “Your plan has ended. You can still finish existing jobs and access your records.” Never display zero data as an error fallback.

19 Reference architecture and repository

ARC01: Use a TypeScript monorepo with apps/mobile for React Native and Expo development builds, apps/portal for Next.js customer pages, apps/admin for staff interface, apps/api for Fastify REST API, apps/worker for background processing and packages/domain for money/schema/state contracts. Use a managed Supabase project for Auth, PostgreSQL and private Storage. Host API, web and worker in the same US region on a managed container platform, default Render paid services. Resend is the default transactional mail provider; RevenueCat manages Apple subscription integration. Use sanitized Sentry error capture and first-party analytics tables. Vendor substitution requires an architecture decision preserving behaviour and acceptance tests.

ARC02: Mobile and public browsers call domain API only; they do not write commercial records directly through Supabase REST. Supabase public Auth endpoints are allowed for owner authentication. Commercial tables reside in a private schema with client grants revoked. API database role is not owner, superuser or BYPASSRLS and uses per-transaction workspace context set only from verified auth; enforce tenant RLS with FORCE ROW LEVEL SECURITY and composite tenant foreign keys. System workers use narrowly scoped functions/roles for queued tasks. Supabase service credentials remain server-only and are never used as routine unrestricted domain authorization.

ARC03: API verifies JWT and workspace membership, opens transaction, sets tenant context, executes authorized command and clears context by transaction-local setting. Connection pooling must not leak tenant context between requests. Customer portal uses a separate scoped API path that invokes narrowly authorized command handlers after token/code checks. Staff endpoints are separately authenticated. Global lookups such as request-token hash resolve only to the minimum request/workspace metadata through restricted database functions; no general cross-tenant list access.

ARC04: A PostgreSQL transactional outbox stores pending PDF, email, export, notification and reconciliation tasks in the same commit as the business event. Workers claim tasks with bounded row locks and SKIP LOCKED, lease for 60 seconds, heartbeat for long jobs, retry with idempotent effect IDs. Do not rely on in-memory queues. Never hold a database transaction open while calling email, storage or billing providers. Generated artifact insert and completion status must be safe if worker crashes between upload and database commit; reconcile orphan blobs by job ID.

ARC05: Use currently supported stable dependency releases at project initialization and pin exact versions/lockfile. Record React Native, Expo, Node runtime, PostgreSQL, PDF renderer, RevenueCat SDK and browser versions in a checked-in compatibility matrix. This PRD intentionally specifies product contracts rather than inventing future SDK patch numbers. CI builds must be reproducible and fail on incompatible lockfile changes. Infrastructure configuration and database migrations live in the owner-controlled repository.

20 Database schema contract

DB01: Use UUID primary keys, timestamptz UTC timestamps, date for commercial local dates, integer/bigint cents constrained to product limits, numeric(12,3) quantities, and jsonb only for immutable typed snapshots or versioned event payloads. All tenant tables include workspace_id UUID NOT NULL, id UUID, created_at, updated_at where mutable, and UNIQUE(workspace_id,id). Every relationship between tenant tables is a composite foreign key (workspace_id, referenced_id). Auth user IDs reference provider identities through a controlled application identity mapping. No cascade deletion of published financial records in ordinary CRUD.

DB02: Unless stated otherwise all listed fields are required; “?” denotes nullable, [] means array, and defaults must be explicit in migrations. created_by is an application actor UUID, with worker actions using a named service actor. Mutable rows use version integer default 1. JSON payloads validate against checked-in schemas on write and read. Enums use constrained text plus migration-defined permitted values so new versions do not silently accept unknown states.

Table	Domain fields beyond common fields
app_users	auth_user_id unique, normalized_email, display_email, status active/suspended/deleting/deleted, last_authenticated_at, deletion_requested_at?, terms_version, privacy_version
workspaces	owner_user_id unique, business_name, legal_name, contact_name, contact_email, contact_phone?, address_json, timezone, currency USD, trade handyman/other, logo_asset_id?, default_tax_bp, default_due_days, default_terms, version
memberships	user_id, role owner, status active; unique workspace/user; v1 exactly one active owner
customers	name, email?, normalized_email?, phone?, billing_address_json?, archived_at?, version
catalogue_items	description, unit, custom_unit_label?, default_quantity, unit_price_cents, discount_cents, tax_bp, archived_at?, version
jobs	customer_id, title, site_address_json?, no_site bool, lifecycle, archived_from_state?, current_quote_id?, active_invoice_id?, scope_version default 0, first_published_at?, entitlement_origin free/trial/paid?, completion_right bool, internal_notes, related_job_id?, version
document_drafts	job_id, kind quote/change/invoice/credit, parent_document_id?, base_scope_version, payload_json, schema_version, draft_state editing/discarded/published, version
documents	job_id, kind, number, revision_no, prior_document_id?, lifecycle issued/accepted/declined/withdrawn/superseded/voided, issued_at, issue_date, due_date?, currency, net_cents, tax_cents, total_cents, snapshot_json, canonical_snapshot_bytes, schema_version, snapshot_sha256, scope_version, void_reason?
document_lines	document_id, position, line_kind source/reduction/credit, source_line_id?, description, quantity?, unit?, unit_price_cents?, discount_cents, net_cents, tax_bp, tax_cents, total_cents, original_source_id?
scope_entries	job_id, source_line_id, accepted_document_id, scope_version, event_kind add/reduce, net_delta_cents, tax_delta_cents; append-only
approval_requests	job_id, document_id, purpose approval/view_only, recipient_name?, recipient_email, token_hash unique, token_key_version, state pending/approved/declined/withdrawn/expired/superseded/revoked, expected_scope_version, expires_at, access_until, token_rotated_at?, decided_at?
approval_challenges	request_id, code_hash, secret_key_version, expires_at, failed_attempts, last_sent_at, consumed_at?
approval_sessions	request_id, session_hash unique, verified_email, expires_at, revoked_at?, token_generation
approval_decisions	request_id unique, document_id, decision approve/decline, signer_name, verified_email, decided_at, snapshot_sha256, consent_version, consent_text, comment?, encrypted_evidence_json?, evidence_key_version?

Table	Domain fields beyond common fields
ledger_entries	invoice_id, type payment/refund/reversal, amount_cents positive, effective_date, method?, reference?, note?, reverses_entry_id? unique, created_by; append-only
credit_allocations	credit_document_id, invoice_line_id, net_credit_cents, tax_credit_cents; unique credit/line
ledger_refund_allocations	refund_entry_id, payment_entry_id, amount_cents positive; append-only, same invoice/workspace
assets	job_id?, draft_id?, visibility internal/customer, bucket_key, upload_state pending/processing/ready/rejected, media_type, source_size, stored_size?, width?, height?, sha256?, rejection_code?, uploaded_by
document_assets	document_id, asset_id, position; append-only after document issue
artifacts	document_id?, export_id?, type original_pdf/status_pdf/receipt_pdf/statement_pdf/export_zip, object_key, sha256, bytes, template_version, generated_at, state ready/failed
document_counters	type quote/change/invoice/credit, next_value; unique workspace/type
job_allowances	free_jobs_consumed default 0, trial_started_at?, trial_ends_at?, trial_jobs_consumed default 0, retained_bytes, version
entitlement_snapshots	user_id unique, provider_customer_id unique, entitlement pro, status, product_id?, expires_at?, will_renew?, verified_at, source_event_id?, environment sandbox/production
provider_events	provider, external_event_id unique per provider, received_at, payload_encrypted?, processing_state, processed_at?, attempts, last_error_code?
outbox_tasks	event_id unique, task_type, aggregate_id, payload_json, schema_version, available_at, lease_until?, attempts, status pending/running/done/dead, effect_key unique, last_error_code?
delivery_attempts	document_id?, request_id?, template_id, recipient_email_encrypted, state, provider_message_id?, effect_key unique, last_event_at, retry_count
idempotency_records	actor_scope, key, route, request_hash, operation_id unique, status, response_code?, response_json?, created_at, expires_at; unique actor_scope/key
audit_events	actor_type, actor_id?, action, entity_type, entity_id, occurred_at, request_id, before_version?, after_version?, reason?, safe_metadata_json; append-only
exports	owner_id, cutoff_at, status queued/running/ready/failed/expired, manifest_json?, expires_at?, error_code?
support_cases	owner_id, category, message, state, content_access_granted_at?, content_access_expires_at?, assigned_staff_id?
analytics_events	event_id unique, pseudonymous_owner_id?, job_id?, event_name, schema_version, occurred_at, received_at, safe_properties_json
staff_users	Global table: auth_subject unique, role agent/supervisor/infra, mfa_required true, status; no commercial tenant access by default
staff_access_grants	staff_user_id, support_case_id, scope_json, reason, approved_by, expires_at, revoked_at?
deletion_requests	owner_id, requested_at, verified_at, status locked/purging/completed/exception, purge_deadline, completed_at?, retained_categories_json?

DB03: Index every list path by (workspace_id, updated_at DESC, id DESC); jobs additionally by (workspace_id,lifecycle,updated_at). Index normalized customer email by workspace, document (workspace_id,kind,number,revision_no), approval request hash, state/expiry, ledger invoice and time, task status/available_at and lease_until. Add a partial unique index on approval_requests(workspace_id,job_id) WHERE state='pending' AND purpose='approval'. A state transition must expire/withdraw the old pending row before creating its successor in one transaction. Add partial uniqueness for the active non-voided final invoice per job, enforced using a dedicated invoice registry or immutable issued row plus current-pointer transaction; do not rely only on UI.

DB04: Immutable-record triggers reject UPDATE/DELETE to commercial payload columns after publication. Status transitions are separate restricted commands and append audit events. Worker/admin roles do not have arbitrary writes to financial tables. Ledger reversal trigger/function checks reference belongs to same invoice/workspace and was not already reversed. Authorization tests must exercise database access paths, not only HTTP handlers.

DB05: Deletion uses a privileged, audited purge workflow outside ordinary application roles. Operational immutable does not mean impossible to delete on an authorized privacy request. Delete links, derived artifacts and owner records consistently; segregate only narrowly necessary legal/security records as section 25 specifies. Backups and restored copies follow the same deletion ledger.

21 API conventions and payloads

API01: Base /v1, JSON UTF-8, HTTPS only. Owner requests use Bearer access token. Public portal uses its HttpOnly scoped session cookie and CSRF token on mutations. Every state-changing owner/public command requires Idempotency-Key UUID and a unique request ID; draft PATCH also requires If-Match numeric version. A different body with a reused idempotency key returns 409 IDEMPOTENCY_MISMATCH. Preserve successful financial operation deduplication permanently through operation_id uniqueness, even after cached responses expire at 30 days.

API02: Success response is {data,meta:{request_id,server_time}}. List data includes items and next_cursor; default 25, max 100, stable descending (updated_at,id), opaque cursor bound to filters. Error is {error:{code,message,field_errors,retryable},meta:{request_id}}. Do not expose stack traces, SQL, provider tokens or tenant existence. Validation is 422, unauthorized 401, permission/entitlement 403, not found 404, stale/state conflict 409, rate limit 429 with Retry-After, asynchronous accepted 202. Retryable 5xx may still represent a committed command; resolve operation before replay.

API03: Fields not listed in a request schema are rejected. The client cannot set calculated totals, tenant ownership, approval state, invoice paid state, server issue time or entitlement. Common pagination query is cursor?,limit?,search?,state?. Dates accept YYYY-MM-DD; timestamps ISO8601 with Z. Currency USD literal. Snapshot responses include server-calculated totals, schema_version, version or revision ID and content digest where applicable.

Draft payload schema

A quote draft body contains job_id UUID, customer_snapshot {name,email?,phone?,billing_address?}, site_snapshot?, public_notes string, terms string, expiry_date date and lines[]. Each line contains client_line_id UUID, description string, quantity decimal string, unit enum, custom_unit_label?, unit_price_cents integer, discount_cents integer, tax_bp integer and asset_ids[]. Internal notes are stored only on job, not this snapshot. Owner business snapshot is applied by server at publish from a reviewed draft preview; if defaults change between preview and publish, 409 PREVIEW_CHANGED requires a new review.

Change draft adds reason string, expected_scope_version integer, additions[] using the line schema, reductions[] {source_line_id,net_credit_cents} and public asset IDs. Direct invoice draft uses normal lines and requires direct_invoice true, issue acknowledgement true, due_date and payment_instructions. Quote-based invoice preview accepts no editable line prices; it is generated from scope. Credit draft contains invoice_id, reason, allocations[] {invoice_line_id,net_credit_cents}. Payment body contains amount_cents, effective_date, method, reference?,note?,confirm_overpayment boolean default false.

Example request

POST /v1/jobs/{job_id}/changes with Idempotency-Key and expected current version creates a draft. A valid body is:

```
{
  "expected_scope_version": 1,
  "reason": "Customer requested replacement of a second handle",
  "additions": [{
    "client_line_id": "11111111-1111-4111-8111-111111111111",
    "description": "Supply and fit second door handle",
    "quantity": "1.000",
    "unit": "item",
    "unit_price_cents": 8500,
    "discount_cents": 0,
    "tax_bp": 0,
    "asset_ids": []
  }],
  "reductions": []
}
```

The server returns a draft ID, version, old total, addition total, reduction total, change total and new total. This does not publish, email or approve the change. A separate publish command binds preview_hash, recipient email and expiry date after owner review. Replacing a still-pending quote also requires replace_pending_request_id and confirmation that its recipient can no longer approve the old version. The UUID is fictional fixture data.

Error code contract

Code	Trigger	UI resolution
VALIDATION_FAILED	Invalid field or amount	Inline fields; keep draft
VERSION_CONFLICT	Stale mutable row	Open S23 with both copies
PREVIEW_CHANGED	Defaults or snapshot differ from reviewed preview	Regenerate preview and require confirmation
SCOPE_CHANGED	Accepted scope version changed	Rebase change draft and review
APPROVAL_PENDING	Another request exists	Open request; wait or withdraw
REQUEST_EXPIRED	Commercial expiry passed	Customer contacts business; owner creates revision
REQUEST_UNAVAILABLE	Revoked/unknown token	Generic unavailable page
ALREADY_DECIDED	Terminal decision already exists	Show decision receipt without new write
UNRESOLVED_CHANGES	Draft/pending changes remain at invoice issue	Resolve each before issue
DOCUMENT_IMMUTABLE	Edit attempted after publication	Offer permitted revision/credit path
CREDIT_EXCEEDS_SOURCE	Reduction/credit exceeds remaining line value	Show server remaining amount

Code	Trigger	UI resolution
REFUND_EXCEEDS_BALANCE	Refund above refundable amount	Show maximum and ledger
ENTRY_ALREADY_REVERSED	Duplicate correction	Show original reversal
ENTITLEMENT_REQUIRED	New-job publication not entitled	S21; preserve draft
QUOTA_EXCEEDED	Trial/fair-use/storage cap	Explain actual cap and next option
ASSET_NOT_READY	Selected photo unfinished/rejected	Retry upload or explicitly remove
PURCHASE_ACCOUNT_MISMATCH	Restore belongs to another app identity	Account sign-in/support flow
OPERATION_PENDING	Command result unresolved	Poll operation; disable duplicate submit
RATE_LIMITED	Abuse/request threshold	Respect Retry-After
ACCOUNT_DELETING	Workspace locked for purge	Deletion status/support only

22 Endpoint inventory

All commands below inherit authorization, idempotency, versioning, limits and errors from section 21. “Owner” means verified workspace owner; “recipient” means valid scoped request session. This inventory is the minimum implementation contract, not permission to expose raw CRUD on immutable tables.

Method and route	Access	Input and result
GET /me	Owner	User/workspace/bootstrap state and entitlement summary
POST /workspace	Owner	Setup fields; create one workspace, memberships and allowances atomically
PATCH /workspace	Owner	Allowed business defaults with If-Match; future drafts only
POST /account/email-change	Owner recent auth	New email; provider verification workflow
GET /customers	Owner	Search/filter/page; own records only
POST /customers	Owner	Customer fields; versioned record
PATCH /customers/{id}	Owner	Mutable contact fields with If-Match
POST /customers/{id}/archive	Owner	archived boolean; referenced records preserved
DELETE /customers/{id}	Owner	Only unreferenced record; otherwise 409
GET /items	Owner	Catalogue search/page
POST /items	Owner	Catalogue fields
PATCH /items/{id}	Owner	Versioned catalogue defaults
POST /items/{id}/archive	Owner	archived boolean
GET /jobs	Owner	Search, state, archive filter, page
POST /jobs	Owner	Client UUID, customer, title, site, mode; create draft

Method and route	Access	Input and result
GET /jobs/{id}	Owner	Overview, permitted_actions, scope and ledger summary
PATCH /jobs/{id}	Owner	Title/internal notes; customer/site only before publication
POST /jobs/{id}/archive	Owner	Archive/restore; no pending request
POST /jobs/{id}/cancel	Owner	Reason; withdraw pending, retain receivable
POST /jobs/{id}/finish	Owner	Verify settlement/credit and close working view
DELETE /jobs/{id}	Owner	Draft-only delete
POST /jobs/{id}/quote	Owner	Create draft or next revision; source document optional
POST /jobs/{id}/changes	Owner	Additions/reductions, scope version; draft
GET /drafts/{id}	Owner	Payload and version
PATCH /drafts/{id}	Owner	Full validated draft payload; If-Match
POST /drafts/{id}/discard	Owner	Mark discarded; preserve audit
POST /drafts/{id}/preview	Owner	Calculate and return preview_hash plus rendered preview data
POST /drafts/{id}/publish	Owner	preview_hash, recipient, expiry, replace_pending_request_id?; snapshot/request, 202 delivery pending
GET /documents/{id}	Owner	Immutable snapshot plus separate live status
GET /documents/{id}/download	Owner	Artifact state or five-minute signed URL
POST /requests/{id}/resend	Owner	Bounded retry without expiry extension
POST /requests/{id}/withdraw	Owner	Reason; terminal withdrawal
POST /requests/{id}/replace-link	Owner recent auth	Revoke tokens/sessions and create fresh access
POST /jobs/{id}/invoice-preview	Owner	Due date/instructions; generated scope and preview_hash
POST /jobs/{id}/issue-invoice	Owner	preview_hash; immutable invoice, completion right check
POST /invoices/{id}/void	Owner	Reason; strict ledger/credit conditions
POST /invoices/{id}/replacement-preview	Owner	Voided source and permitted corrected identity fields
POST /invoices/{id}/issue-replacement	Owner	Validated preview_hash, new official number
POST /invoices/{id}/credits/preview	Owner	Allocations/reason; calculated credit
POST /invoices/{id}/credits	Owner	preview_hash; issue credit and notify
GET /invoices/{id}/ledger	Owner	Entries/reversals and derived balance
POST /invoices/{id}/payments	Owner	Manual payment body; posted ledger
POST /invoices/{id}/refunds	Owner	Manual refund body; posted ledger
POST /ledger/{id}/reverse	Owner	Required reason; one reversal only
POST /documents/{id}/send	Owner	Existing recipient or verified replacement read-only link, template

Method and route	Access	Input and result
POST /assets/upload-url	Owner	Job/draft, filename display only, type/bytes; signed upload
POST /assets/{id}/complete	Owner	Upload finished; enqueue validation
GET /assets/{id}	Owner	Processing state and authorized thumbnail if ready
DELETE /assets/{id}	Owner	Unpublished unused asset only
POST /subscription/trial	Owner	Explicit start acknowledgement; once only
GET /subscription	Owner	Verified entitlement, trial and usage
POST /subscription/reconcile	Owner	Provider transaction hint; fetch authoritative state
POST /webhooks/revenuecat	Provider auth	Durable event receipt, asynchronous processing
POST /webhooks/email	Provider signature	Delivery event deduplication
POST /exports	Owner recent auth	Cutoff request; 202 export ID
GET /exports/{id}	Owner	Status/manifest and secure download if ready
POST /account/deletion	Owner recent auth	Confirmation phrase; lock/purge request
GET /account/deletion	Owner limited session	Deletion state and timeframe
POST /support/cases	Owner	Category/message and optional content-access grant
GET /operations/{id}	Same initiating actor	Pending or durable result for timed-out command
POST /analytics/batch	Owner	Allowlisted events, max 50; deduplicate event IDs
POST /portal/exchange	Request token	Scope cookie; no private commercial data yet
POST /portal/code/send	Preverified session	Request OTP to bound email; generic response
POST /portal/code/verify	Preverified session	Code; grant one-hour recipient session
GET /portal/document	Recipient	Bound snapshot and allowed action state
POST /portal/decision	Recipient plus CSRF	decision,name,consent_version,snapshot_hash; durable receipt
GET /portal/receipt	Recipient	Existing decision and receipt artifact
GET /portal/download	Recipient	Authorized original/status artifact link
POST /portal/report	Scoped or preverified session	Abuse reason; bounded staff ticket
GET /admin/cases	Staff	Metadata-only case search

Method and route	Access	Input and result
POST /admin/cases/{id}/access	Supervisor	Owner grant plus reason; bounded access session
POST /admin/requests/{id}/revoke	Supervisor	Incident reason; no financial mutation
POST /admin/tasks/{id}/retry	Staff	Retry idempotent failed task; reason logged

API04: Direct-invoice editing uses the generic draft routes: POST /jobs creates mode `direct_invoice` and its initial draft; preview uses `/drafts/{id}/preview` and issue uses `/jobs/{id}/issue-invoice` with that draft `preview_hash`. GET `/jobs/{id}` includes its drafts/document summaries; a separate document list endpoint is unnecessary in v1. After issue, all amount changes follow the credit/replacement rules.

API05: Read-only invoice/credit access shares the request/session machinery with approval requests but purpose is `view_only`, with state pending meaning access enabled and no decision allowed. Do not expire this state through the approval-expiry worker; use access expiry. Its GET portal response has `allowed_actions=[download,report]` and no approval controls. POST decision returns 403 for this purpose. Persist purpose approval/view_only on approval_requests; approval-only pending uniqueness excludes view_only requests. All portal code routes validate purpose. For view_only requests `expires_at` controls access, default 90 days; owner may issue a fresh request. This distinction must be in migration/schema and OpenAPI, not inferred from UI. For approved or declined requests, GET portal/receipt returns the decision; for view_only requests it returns document access metadata rather than inventing a consent receipt.

23 Transaction algorithms

TX01 Publish: verify owner and draft version; lock job, allowances and current request; verify completion entitlement or consume available new-job slot; reject any unrelated pending approval; if `replace_pending_request_id` explicitly identifies the current quote request with the expected version, supersede it atomically and invalidate its sessions; recalculate using server domain package; match preview hash; validate ready assets; allocate number/revision; insert immutable snapshot/lines/assets; create tokenized request and durable outbox; mark draft published and job active; commit. Return document/request IDs immediately. Any precommit failure rolls back all side effects. Generate no external email inside transaction.

TX02 Approve: verify recipient session and CSRF; lock request and job; compare now with expiry, state, document hash and expected scope version; verify source reduction caps; insert unique decision; apply append-only scope entries; mark quote/change accepted; increment `scope_version`; commit receipt/notification tasks with the decision. The online response comes from committed data. Never accept after expiry based on the time the customer first opened the page.

TX03 Invoice issue: lock job; verify current scope/preview hash and absence of pending/unresolved changes; validate direct-mode exception if used; enforce one active invoice; insert snapshot/lines and number; set active invoice pointer/lifecycle; append audit/outbox; commit. A response timeout is recovered through `operation_id`. Worker failure cannot permit a duplicate invoice.

TX04 Credit/payment/refund/reversal: lock invoice; load effective ledger and cumulative credits; validate caps and transaction-specific conditions; insert append-only entry or credit snapshot; recompute balance; append audit/outbox; commit. Concurrent commands must serialize on the same invoice and must never both spend the same remaining credit/refund capacity.

TX05 Billing: authenticate and store webhook event by provider/external ID; acknowledge only after durable persistence. Worker retrieves current provider subscriber state, validates configured environment and user

mapping, compares effective dates, updates entitlement snapshot and audit. If provider unavailable, retry without revoking previously valid access prematurely. Nightly reconciliation repairs missed events.

TX06 Public link compromise: revoke request access tokens and sessions, block future decisions on compromised pending request, preserve already committed decisions, issue fresh request requiring verification and review when appropriate. Never erase historical approval evidence as a security shortcut. Notify owner through verified channel and attach incident ID.

24 Security and abuse controls

SEC01: Enforce HTTPS/TLS, secure cookies, HSTS on owned web domains, exact-origin CORS and a restrictive Content Security Policy. Do not place authentication tokens in localStorage, URLs, analytics properties or error reports. Keep public review pages free of third-party scripts. Escape all user content; render plain text only. No remote user HTML, arbitrary URL PDF fetches or externally embedded images, preventing PDF-renderer SSRF and data leakage.

SEC02: Verify every owner object, composite relation, selected source line and asset belongs to the same workspace/job. A valid ID from another tenant must never be usable as a source_line_id, attachment, invoice or export. Rate-limit owner API to 120 requests/minute/user with separate publication limit 20/hour/workspace and daily email limit 100 free/trial, 500 paid, plus provider safeguards. Completion rights do not bypass abuse limits; legitimate higher volume uses support rather than silent record lockout. Public OTP limits are separately stricter.

SEC03: File validation runs in isolated worker resources, with decode memory/time limits. Product infrastructure rejects upload bodies larger than the maximum before processing. Server-generated artifact keys are not user-controlled. Use randomized IDs and opaque tokens rather than sequential document numbers as access credentials. Publish webhook routes have signature/secret checks, replay deduplication, size limits and fail-closed processing.

SEC04: Source control contains no service keys, signing certificates, database passwords or actual customer samples. Secret values live in owner-controlled managed secret stores and are scoped by environment. Rotate on staff departure or suspected compromise. Public mobile configuration contains only intended public keys, not Supabase service role, RevenueCat server key, email API secret or PDF/download signing keys.

SEC05: Support console is behind separate staff authentication, MFA and least privilege. Content access requires owner grant for one case, expires after 24 hours and is revocable. Staff action requires reason and immutable audit; export and financial editing remain prohibited. Break-glass operational access requires incident record and two-person authorization in production, with owner notification where appropriate. This is an engineering control to implement, not a request for the product owner to approve ordinary document creation now.

SEC06: Provide Report a problem on customer pages for fraudulent/abusive requests. Repeated complaints suspend outbound sending after review; affected owners retain authenticated read/export subject to genuine security restrictions. No searchable public directory of businesses or documents. Domain/content moderation is bounded to private transactional usage; user data is not licensed for model training or advertising.

SEC07: Before launch run automated dependency and secret scanning, database isolation tests, API authorization tests, upload attack fixtures, portal token/CSRF tests and a focused independent security review of approvals/billing/tenant boundaries. Resolve all exploitable critical/high findings before production. Record residual low-risk issues with owner and due date; do not describe scanning alone as penetration testing.

25 Privacy retention and account deletion

PRV01: Publish a privacy notice describing owner/contact details, customer records, photos, commercial documents, approval verification/evidence, service providers, retention, deletion and international access by support. Customer information is processed to provide the operator's service; actual legal roles and contractual wording require counsel review for the chosen entity and markets. The technical choices here are policy requirements, not a declaration of legal compliance.

PRV02: Default retention while an account exists: commercial documents, receipts, ledger and business history persist until account deletion; do not promise a legally sufficient archival term. Retain operational delivery logs and error logs 30 days, security logs 90 days, raw approval IP/user-agent evidence 90 days then delete it, and core decision receipt fields for as long as the associated record. Minimized aggregate product metrics may persist 13 months with per-user deletion capability. Suppress content from telemetry at ingestion. Owner exports are the customer's archival mechanism; app account existence is not a guaranteed permanent archive.

PRV03: Account deletion is available inside Settings, not only by emailing support. Reauthenticate within five minutes, display scope and consequences, offer export, and require typing DELETE. Do not force export, subscription cancellation or a support call before accepting deletion. Apple subscription cancellation is separate; show Manage subscription and explain that deleting the account does not itself guarantee cancellation. Never claim the app can cancel an Apple subscription by deleting a database row. Source SREF02 informs this flow.

PRV04: On confirmed deletion, immediately revoke owner and customer access, prevent new writes/sends, cancel pending approvals and remove the account from analytics collection. Start purge within 24 hours and complete live app-record/object deletion within 30 days. Customer live links become unavailable; users must be warned that recipients will lose future portal access. Preserve only specifically documented records needed for tax, fraud, disputes or other valid retention obligations in a restricted separate store with category, basis, deadline and authorized role. Legal reviewer supplies those categories before launch; there is no default blanket exemption retaining every invoice.

PRV05: Managed database backups expire within the configured 35-day maximum. A deletion ledger is applied before any restored system is exposed to users, so deleted records cannot reappear. Purge Storage objects separately; do not assume database backup or row deletion deletes photos/PDFs. Request deletion from Auth, RevenueCat, error/analytics and email systems where applicable; distinguish unavoidable provider financial retention from app data. Verify provider capabilities and publish accurate retention wording before release.

PRV06: The account remains locked during purge; no self-service cancellation of deletion after confirmed irreversible action in v1. Provide a deletion receipt before credentials are removed. The narrow fraud-prevention record for previously used trials is a keyed pseudonymous identifier only where reviewed and necessary, retained at most 12 months; it is still personal data, not anonymous. Do not secretly retain full profiles to prevent another free trial. The business accepts some trial abuse rather than violating published deletion commitments.

26 Accessibility performance and resilience

NFR01: Target WCAG 2.2 AA for the customer portal and corresponding native accessibility quality. Support VoiceOver labels, headings, error announcements, logical focus, text alternatives for meaningful images, system font scaling, keyboard operation on web, reduced motion and visible focus. Approval and payment actions must remain usable without colour perception or precise gestures. Photos have owner-entered optional captions; the scope text must contain necessary work details so photos are not the only accessible description. Source SREF05 is the normative web accessibility reference.

NFR02: Test on iPhone SE third generation and a current standard iPhone at minimum, with supported older OS and latest stable iOS, low storage, airplane mode and network interruption. Browser matrix: Safari on supported

iPhone OS, Chrome Android current and previous major, Chrome/Edge desktop current and previous major, Safari macOS current. No iPad-specific operator layout is promised, but store compatibility mode must not break core use if distributed there.

NFR03: Under a reference load of 100 concurrent active owners, 20 recipient sessions and 10 workers, target API read p95 below 500 ms and transactional write p95 below 800 ms excluding external-provider latency, measured server-side. Customer portal meaningful content after verification below 2.5 seconds on a representative 4G connection. PDF ready p95 under 15 seconds for 20 lines/two images, under 60 seconds for the supported maximum. These are acceptance targets to test, not measured present performance.

NFR04: Target 99.9 percent monthly API/portal availability excluding preannounced maintenance, with measured synthetic health checks. Use managed database point-in-time recovery with recovery point objective at most 15 minutes and recovery time objective at most four hours for regional data corruption recovery. Configure a provider plan that can support those objectives. Encrypted object versions/backups require separate restoration verification. A successful database backup alone does not meet recovery acceptance.

NFR05: Owner draft changes must survive app termination after the local Saved indicator. Committed server records must survive API/worker restarts. PDF/email failures are visible and retryable. If database unavailable, do not return invented empty lists or falsely successful publications. If storage unavailable, block publication involving unfinished attachments while preserving draft text. If billing unavailable, preserve last verified entitlements to known expiry and show access-check status.

NFR06: Timeout defaults: mobile reads 15 seconds, commands 20 seconds, public decision 20 seconds, provider calls 10 seconds with bounded retries outside business transactions. Background tasks use workload-specific limits: PDF 60 seconds, individual image 30 seconds, export batch five minutes with resumable checkpoints. Never auto-retry a non-idempotent external effect without resolving its effect key.

27 Analytics instrumentation

ANA01: Implement a first-party event schema with event_id UUID, event_name, schema_version, occurred_at, received_at, pseudonymous owner ID, optional opaque job ID, app version, platform and allowlisted properties. Never send customer names/emails, addresses, note text, invoice descriptions, approval names, URLs/tokens, photos or payment references. Pseudonymous owner IDs are random mapped identifiers, not raw email hashes. Exports/deletion must account for the mapping.

Event	Emit when	Allowed properties
signup_verified	Provider verification and app bootstrap succeed	acquisition_source self_reported/unknown, app_version
onboarding_completed	Required setup saved	trade, setup_duration_bucket
job_created	Server draft job created	mode quote/direct
document_published	Snapshot commit succeeds	kind, entitlement_origin, line_count_bucket
request_delivery_result	Verified provider event	result, template_id
approval_completed	Server decision commits	approve/decline, document_kind, elapsed_bucket
change_started	Change draft created	addition/reduction/mixed
invoice_issued	Invoice transaction commits	quote_based/direct, has_changes boolean
payment_recorded	Manual payment commits	partial/full/overpaid, no amount or reference
trial_started	Server trial record first set	remaining_free_slots

Event	Emit when	Allowed properties
paywall_viewed	S21 visible	entry_point, product_ids, experiment_id?
purchase_verified	Server entitlement reconciled	product_id, initial/renewal, store_environment
subscription_expired	Verified state loses Pro	known_reason enum, no raw provider payload
export_completed	Bundle ready	size_bucket, job_count_bucket
sync_conflict	Server draft conflict	resource_kind, client_version
support_opened	Case created	category

ANA02: Server business events are authoritative and deduplicated; clients do not emit a second purchase_verified or document_published. Offline UI events batch at most 50 and expire after seven days. Conversion reports exclude demo/test/staff accounts and sandbox transactions. Define cohort start as signup for activation and first verified paid transaction for subscription retention. Store invoices paid by operators' customers are unrelated to app subscription revenue.

ANA03: V1 acquisition reporting uses campaign-level spend imports and self-reported source with unknown bucket. No deterministic install-to-ad promise. Apple's privacy-preserving attribution integration may be added only after a separate validated specification; do not install ad SDKs merely to satisfy an assumed metric. Experimentation is limited initially to pre-paywall copy and creative entry points, never calculation, customer consent or entitlement correctness. No simultaneous untracked pricing tests.

28 Acceptance test matrix

Each test is a developer/QA requirement, not a claim that the unbuilt app has passed. Link automated tests and manual evidence to IDs in the release tracker. Unit tests cover domain functions; integration tests use real database constraints; end-to-end tests use sandbox providers and controlled test mailboxes; physical device tests cover OS lifecycle and billing.

Test	Scenario	Required outcome
QA01	New owner verifies email and sets up workspace	Exactly one workspace, defaults and allowances; restart resumes correct next step
QA02	Wrong/expired owner code and resend flood	Generic error, cooldown and provider limits; no account enumeration
QA03	Two owners request each other's customer/job IDs	404 and no data leakage in API, storage or logs
QA04	Cross-tenant line/asset ID injected into own draft	Rejected before snapshot publication
QA05	App terminated after local save	Draft reappears exactly as saved
QA06	Network removed during typing/publish	Local persistence visible; publish blocked or operation resolved accurately
QA07	Two phones edit same monetary field	409 conflict; both copies recoverable; no last-write-wins
QA08	Replay draft operation 20 times	One applied mutation and stable resulting version
QA09	Publish timed out after commit and retried	One number, job slot, request and outbox effect

Test	Scenario	Required outcome
QA10	Fourth new free job	Paywall with preserved draft; first three can still complete
QA11	Trial activation replay/reinstall/email change	Original expiry/cap retained; no accidental billing
QA12	Trial expires during pending customer approval	Customer can approve; owner can finish existing job
QA13	Paid owner downgrades with existing jobs	Completion/export work; new-job gate reflects remaining free slots
QA14	Attempt reset customer/site of published job	Blocked; user must create new job
QA15	Owner edits catalogue price and business address	Issued documents unchanged; new draft uses reviewed defaults
QA16	Publish replacement while old request pending	Old superseded, new pending; one pending index holds
QA17	Old recipient follows replaced link	Cannot decide; correct unavailable/superseded message
QA18	Link forwarded to unrelated person	Cannot view scope or approve without bound-email code
QA19	OTP brute force and resends	Attempt/rate limits; no raw codes logged
QA20	Approve exact request twice concurrently	One decision and one scope application
QA21	Approve and withdraw race	One valid terminal outcome; loser receives conflict
QA22	Approve one millisecond after expiry	Rejected regardless of worker display lag
QA23	Request opened before expiry then approved after	Rejected; no cached approval authority
QA24	Consent checkbox unticked or hash altered	Approval rejected; full reviewed version required
QA25	Declined quote revised and resent	New revision; previous decision preserved
QA26	Accepted quote edited directly	Immutable write rejected in API and database
QA27	Addition then approved reduction	Scope ledger accurate; original source and approvals visible
QA28	Two reductions exceed remaining source	Later invalid transaction rejected atomically
QA29	Reduction fixtures F05 and F06	Exact cumulative tax and cap results
QA30	F01 to F12 across mobile/server/PDF	Exact matching cents and balances
QA31	Invoice attempted with pending or draft change	Block with actionable resolution
QA32	Invoice issue raced with change publish	Serialization prevents inconsistent scope/invoice
QA33	PDF worker crashes after invoice commit	Invoice stays issued, one artifact eventually ready, no duplicate number
QA34	Provider email timeout after acceptance	Reconcile effect before retry; no blind duplicate send
QA35	Permanent bounce	Not shown delivered; owner correction path and suppression
QA36	Full approved scope reduced to zero	Zero invoice, settled by reduction, no fake payment
QA37	Manual partial/overpayment	Derived balance and explicit overpayment confirmation

Test	Scenario	Required outcome
QA38	Credit after full payment	Negative balance labelled refund due; no money moved
QA39	Refund exceeds available balance	Rejected; partial legitimate refund works
QA40	Reverse payment once and retry	One reversal; resulting effective ledger correct
QA41	Reverse payment with effective dependent refund	Block or require correction per ledger rules
QA42	Void invoice with payment or issued credit	Rejected; historical record intact
QA43	Unpaid invoice void then replacement	New number and link to voided original; only one active invoice
QA44	Direct invoice without quote	Clear no-prior-approval label, ordinary invoice calculation
QA45	Monthly purchase canceled/pending/success	No false Pro; success verified on server
QA46	Purchase SDK success with backend unavailable	Pending verification UI, receipt context retained
QA47	Restore same Apple receipt under another owner	No entitlement theft/transfer; recovery message
QA48	Billing events duplicated/out of order	Current provider state wins; stable entitlement
QA49	Cancellation before expiry	Access until actual expiry; no premature block
QA50	Refund/revocation and grace period	Correct verified entitlement, historical access preserved
QA51	Trial and purchase concurrently	No double trial allocation; paid access overrides as verified
QA52	Malformed/oversize/EXIF/GPS photo upload	Reject or sanitized accepted image; no metadata leak
QA53	Signed URL expired or wrong tenant	Access denied without revealing data
QA54	PDF 100 lines, long names, 10 images	Legible multipage layout, no clipping, totals and numbering correct
QA55	Unicode names and CSV formula text	Correct display; safe export in spreadsheet applications
QA56	Export after subscription expiry	Complete authenticated export with manifest and checksums
QA57	Account deletion with active subscription	Accept deletion, explain billing separately, revoke links and purge
QA58	Restore backup containing deleted owner	Deletion ledger reapplied before exposure
QA59	Staff case access without owner grant	Metadata only; content denied and attempt audited
QA60	Staff grant expires/revoked	Content access stops immediately at next request
QA61	VoiceOver/Dynamic Type/keyboard/zoom	All core flows usable, no hidden actions or unreadable totals
QA62	Low disk during draft save	Explicit save failure, do not display Saved
QA63	Token in URL/error/analytics/referrer attempts	Redacted or absent; no raw secret exposure
QA64	Daylight saving and timezone changes	Stored expiry invariant; due-date behaviour correct
QA65	Worker double claim/lease expiry	One effective external task; recovery after crash

Test	Scenario	Required outcome
QA66	Database backup and object restore drill	RPO/RTO measured; artifacts usable, hashes verified
QA67	Maximum supported load	NFR03 measured; backlog drains without lost tasks
QA68	No configured production secrets	CI/deployment fails safely; no demo keys or placeholder legal text shipped

29 Developer delivery and definition of done

DEL01: Deliver an owner-controlled repository with complete source, dependency lockfiles, environment templates containing names only, reproducible local setup, mobile build configuration, API/worker Dockerfiles, migrations, seed fixtures and infrastructure configuration. No proprietary contractor account may be the sole owner of the code, store listing, domain, billing project or production database. Document licences for fonts, icons and dependencies.

DEL02: Deliver executable OpenAPI 3.1 schemas for every endpoint, request/response examples and documented error codes matching this PRD; generate or validate mobile API client types against them. Deliver an entity relationship diagram, migration order, schema constraints and query/index performance notes. The table in section 20 is a design contract; production SQL must enforce it with tests, not just document it.

DEL03: Deliver editable interface source, all S01 to S28 states, responsive portal frames, design tokens, icons, launch assets and a clickable main-flow prototype. Implement supplied copy with localization keys, not scattered literals. QA must verify the implemented screens against that approved design output. No competitor assets, copied brand identity or scraped protected illustrations.

DEL04: Each feature ticket is done only when domain tests, authorization checks, loading/empty/offline/error states, accessibility labels, analytics event if required, structured operational logs, migration rollback/forward compatibility and documentation are included. Business owner acceptance is needed for product outcome, not for every code line. Do not close tickets with “backend later” when the flow appears interactive but is a mock.

DEL05: Final handover includes staging URL, TestFlight build, test account procedure, customer approval test mailbox workflow, QA matrix evidence, release notes, app-store submission package, runbooks, backup drill evidence, security findings disposition, known limitations and a recorded demonstration of the complete real sandbox journey. Supply a deployment and credential-rotation walkthrough. Secrets are transferred securely, never pasted into documentation or chat.

30 Build sequence and engineering gates

Stage	Scope and dependencies	Exit evidence
0 Foundation and risk spikes	Validate customer workflow; prove encrypted offline storage, provider login, receipt restore mapping and PDF generation on real iPhone	Written architecture decisions, prototype, sandbox screenshots, testable risk resolution
1 Identity and drafts	Workspace isolation, customers/items, jobs, local save/version conflicts	QA01 to QA08 and isolation suite; no publication until persistence works
2 Quotes and approvals	Snapshot schema, hashing, pending uniqueness, token/code portal, email outbox	Full owner/customer quote journey, race tests QA16 to QA26
3 Changes and invoicing	Add/reduce scope, financial module, invoice issue/PDF, ledger/credits	All money fixtures plus invoice/ledger QA; no mutable accepted totals

Stage	Scope and dependencies	Exit evidence
4 Billing and lifecycle	Free/trial slots, completion rights, RevenueCat reconciliation, export/deletion	Sandbox billing/restore and deletion tests; account/receipt mapping documented
5 Hardening and pilot	Security review, device/accessibility tests, performance/restore drill, limited pilot	No critical/high defects, end-to-end evidence, actual pilot feedback
6 Production release	Business inputs complete, store package, phased rollout, monitoring/on-call	Release checklist signed, rollback tested, support staffed

Indicative effort: 10 to 14 calendar weeks for two experienced engineers with part-time design, QA and product oversight, after quick technical spikes. This is a planning range, not a fixed bid or guarantee. It is longer than a bare invoice-generator MVP because offline recovery, secure approval, corrections and production operations are real work. Estimate each stage in the team's own backlog before committing a delivery date. Scope cuts must preserve immutable money and approvals; defer secondary UI polish or a platform instead.

31 Operations monitoring and incident response

OPS01: Maintain development, staging and production as separate Auth, database, storage, email and RevenueCat environments. Sandbox purchases never grant production entitlement. Staging sends only to allowlisted test domains/mailboxes and visually watermarks every generated document TEST. Never copy production personal data into development; seed fictional fixtures. Production migrations require a backup checkpoint and tested forward-compatible rollout.

OPS02: CI gates include lint/types, domain/financial tests, API contract validation, migrations on clean database, tenant isolation tests, dependency/secret scans, worker idempotency tests and a signed mobile build smoke test. Staging end-to-end tests run on every release candidate. Use feature flags to disable new publication or billing initiation during incidents without disabling read/export/known-safe completion. Do not use remote code changes to bypass applicable store review.

OPS03: Log structured request_id, operation_id, tenant pseudonym, endpoint, duration, status/error code and worker effect ID. Redact authorization headers, cookies, token fragments, emails, commercial text, attachments and payment references before logs leave the process. Sentry session replay and screenshots are disabled on all screens. Dashboard charts: API error rate and latency, queue age, publication-to-PDF time, email failures, duplicate command rate, billing reconciliation age, backup freshness, crash-free sessions and storage growth.

OPS04: Alert on any detected cross-tenant access, snapshot hash mismatch or negative source balance immediately; on-call treats it as severity 1. Alert if p95 write latency exceeds two seconds for ten minutes, API 5xx exceeds two percent for five minutes, oldest normal task age exceeds ten minutes, billing reconciliation is stale over one hour for repeated failures, or backup success is absent beyond scheduled window. Route alerts to an actually staffed on-call channel; a dashboard alone is not incident response.

OPS05: Severity 1 includes data disclosure, corrupted issued documents or widespread unavailable approvals. Target acknowledgement within 30 minutes during production coverage, disable the unsafe write path, preserve evidence and open incident record. Severity 2 includes partial email/PDF/provider outage without lost commitments; acknowledge within four business hours. Publish status updates only with verified facts. Data breach communications follow reviewed obligations and entity-specific process; do not invent universal notification deadlines.

OPS06: Restore runbook: isolate affected environment; establish corruption cutoff; restore database to new instance; restore matching object versions; replay deletion ledger and verified immutable operation records;

compare snapshot/PDF hashes and counts; reconcile subscription state and unresolved outbox effects; run isolation/ledger smoke tests; switch traffic only after checks. Do not replay email tasks blindly from old backups. Test this runbook before launch and quarterly afterward.

OPS07: Deploy API backward-compatible with previous mobile version, then worker/portal, then mobile. Add nullable/defaulted fields before requiring them; retire fields only after minimum-version policy and adoption review. API supports current and prior app release for at least 90 days. Critical security minimum-version block may require update but should provide secure export/support access. Rollback cannot undo committed financial history; repair with forward migrations and approved correction commands.

OPS08: Scheduled jobs: poll outbox continuously; expire pending requests every minute while endpoints enforce exact expiry; cleanup stale uploads hourly; reconcile paid entitlements nightly; retry bounded reconciliation backlog every five minutes; generate trial reminders once two days before expiry; purge exports daily; apply retention/deletion daily; verify backup status daily; calculate deidentified cohort reports daily. Every schedule has a unique execution key and observability.

32 Store submission and release acceptance

REL01: Apple release must use the then-current required SDK/toolchain, accurate app privacy labels and SDK privacy manifests/required-reason declarations where applicable. Check current Apple guidelines before submission, especially digital-feature billing, subscription presentation, privacy and account deletion. This specification chooses Apple in-app subscriptions and does not depend on region-specific external-purchase exceptions. Physical job payments are not processed by the app. Source SREF01 supplies the policy baseline; passing tests cannot guarantee store approval.

REL02: Prepare owned developer account, bundle ID, signing, distribution certificates, subscription group/products, privacy policy URL, terms URL, support URL/contact, store description, screenshots from the actual production-equivalent build, age-rating answers and review notes. Review must have a fully functional controlled demo mode or dedicated review identity and documented access to test inboxes without a global authentication bypass. Explain that the 14-day app trial never automatically charges and that customer approval is a separate web flow.

REL03: Initial rollout is invitation-only TestFlight with 15 to 25 real pilot operators, followed by limited public release and staged increase. Test data clearly separated from real customer records. Pilot terms and live email expectations must be clear. No paid acquisition at scale until real publication, approval, invoice, cancellation and recovery flows have passed. Rollout changes are operational releases, not automatic permission to spend advertising budget.

Release checklist

- Every MUST requirement mapped to an implemented ticket and passing test/evidence.
- All 68 QA scenarios executed; financial, isolation and approval concurrency tests automated.
- No open severity 1/2 defects or exploitable high/critical security findings.
- Production configuration complete; legal/support pages contain no placeholders.
- Apple sandbox purchase, renewal, grace, cancellation, refund, restore and account-mismatch flows demonstrated.
- Free/trial counters and completion rights verified across reinstalls and two devices.
- PDF/receipt output checked at supported maximum sizes; all money fixtures match.
- Privacy deletion, export and backup restoration drills completed with recorded evidence.
- Actual rate limits, object retention and backup plans configured rather than merely described.
- Support console permissions and on-call coverage verified; incident drills rehearsed.

- Current store policy and third-party licences reviewed; no unsupported marketing promises.
- Owner receives repository, infrastructure/store ownership and secure credential inventory.

33 Configuration and service provisioning

All values below are deployment configuration or secrets, not hardcoded assumptions scattered through the code. Empty mandatory production values prevent startup or deployment. UI-facing prices come from StoreKit. Nonsecret operational limits are versioned centrally and audit changes.

Configuration	Classification	Requirement
APP_ENV	Public config	development, staging or production; immutable per deployment
PUBLIC_APP_NAME and SUPPORT_URL	Public config	Final brand and monitored support destination
OWNER_APP_BUNDLE_ID	Public config	Owner-controlled identifier; same across signing and billing setup
API_BASE_URL and PORTAL_ORIGIN	Public config	HTTPS owned domains; exact allowlists
AUTH_PROJECT_URL and AUTH_PUBLISHABLE_KEY	Intended public config	Environment-specific Supabase Auth
AUTH_ISSUER and AUTH_AUDIENCE	Server config	Strict expected JWT values; no accept-any issuer
DATABASE_URL_API	Secret	Restricted app role; separate migration/worker roles
DATABASE_URL_MIGRATIONS	Secret	Deployment-only role, unavailable to mobile/runtime
STORAGE_SERVICE_KEY	Secret	Server-only, restricted use for authorized objects
APPROVAL_TOKEN_HASH_KEY and OTP_HASH_KEY	Secret	Independent versioned keys; rotation procedure
APPROVAL_EVIDENCE_ENCRYPTION_KEY	Secret	Managed key, restricted decrypt role and rotation
REVENUECAT_PUBLIC_IOS_KEY	Intended public config	Correct project/app mapping
REVENUECAT_SECRET_KEY and WEBHOOK_AUTH_SECRET	Secret	Server API and webhook auth; separate environment
MONTHLY_PRODUCT_ID and ANNUAL_PRODUCT_ID	Public config	Recorded identifiers in one subscription group
EMAIL_API_KEY and EMAIL_WEBHOOK_SECRET	Secret	Resend transactional environment
EMAIL_FROM_DOMAIN	Server config	Verified SPF/DKIM/DMARC; no arbitrary From
ERROR_REPORTING_DSN	Public config	Sanitized events; replay/screenshots disabled
STAFF_AUTH_CONFIG	Secret/config	MFA and allowed staff roles; no public staff signup
BACKUP_RETENTION_DAYS	Server config	At most 35 for this policy; PITR plan enabled
FEATURE_NEW_PUBLICATION and FEATURE_PURCHASES	Server flags	Emergency stop; independently preserve safe reads
LIMITS_VERSION	Server config	Caps and intervals from this PRD with audit history

Initial recurring infrastructure budget is a planning allowance of USD 250 to 750 per month for properly provisioned managed services and staging, excluding staff, media and store/revenue-based fees. It is not a vendor quotation. Engineering must obtain current service estimates at provisioning, choose paid plans meeting backup/availability requirements, and report actual cost per active owner monthly. Per-user contribution assumptions from the venture analysis are targets, not permission to omit required reliability services.

34 Business inputs and decision register

These are the only unavoidable owner-supplied launch inputs. Developers can build with explicit non-production defaults now; they must not manufacture legal identities, account access or approvals. Scope choices already made in this PRD do not require repeated clarification.

Input	Responsible party	Development default and production gate
Final name and trademark review	Product owner	Job to Invoice working title; production brand clearance before listing
Contracting legal entity and address	Product owner with counsel	Fictional staging entity only; real details mandatory on terms/privacy/store
App Store account ownership	Product owner	Team sandbox setup; owner-controlled account before production signing
Domain and support mailbox	Product owner	Local/staging host; owned verified domain and monitored mailbox before live email
Commercial terms and approval wording review	Product owner with counsel	Supplied neutral acknowledgement in staging; jurisdiction/trade review before enforcement claims
Privacy notice and restricted-retention categories	Product owner with privacy counsel	Implement stated minimal defaults; no blanket commercial-record retention exception
Vendor accounts and payment method	Product owner	Test accounts; production ownership and billing before provisioning
Support staffing and incident contacts	Product owner/operations lead	Named test contact; actual monitored support/on-call before public rollout
Final selected dependency matrix	Engineering lead	Risk spike picks supported stable versions, pins and records before feature build
Validated target user and pricing	Product owner	US solo handyman, USD 19.99/149.99 hypotheses; change through recorded product decision

Changes to launch geography, tax engine, Android launch, team roles, payment collection, signature-provider requirements or progress billing require a scoped revision and cost estimate. Do not expand them informally through customer-support requests. Maintain a decision log with ID, date, decision, evidence, owner, impacted requirement/test IDs and migration implications.

35 Primary sources and accuracy boundaries

The workflow and architecture above are proposed implementation requirements. External sources below were checked on 11 September 2026 for platform and technical constraints. Vendor documentation changes; the

engineering lead must recheck SDK/policy details at dependency selection and store submission. Public sources do not validate market demand for this exact workflow.

SREF01 Apple App Review Guidelines — digital billing, subscriptions, privacy and review preparation.

<https://developer.apple.com/app-store/review/guidelines/>

SREF02 Apple Offering Account Deletion in Your App — account deletion must be designed as a real in-app user flow; subscriptions require separate handling. <https://developer.apple.com/support/offering-account-deletion-in-your-app/>

SREF03 RevenueCat Webhooks — provider event integration and server-side handling.

<https://www.revenuecat.com/docs/integrations/webhooks>

SREF04 RevenueCat Identifying Customers — stable app-user identities and restore/transfer configuration.

<https://www.revenuecat.com/docs/customers/identifying-customers>

SREF05 W3C Web Content Accessibility Guidelines 2.2 — portal accessibility acceptance reference.

<https://www.w3.org/TR/WCAG22/>

SREF06 Supabase JWT Guide — supported JWT verification and key handling.

<https://supabase.com/docs/guides/auth/jwts>

SREF07 Supabase Row Level Security — grants, roles and database isolation must be configured and tested.

<https://supabase.com/docs/guides/database/postgres/row-level-security>

SREF08 Expo SQLite Documentation — persistence/build capabilities, including encrypted-storage support to validate in the chosen native build. <https://docs.expo.dev/versions/latest/sdk/sqlite/>

The preceding venture recommendation selected the invoicing category using reported competitor evidence. Its revenue estimates, acquisition figures, conversion assumptions and retention scenarios are not implemented product facts and must not appear as customer-facing claims. No guarantee of payment, recovered revenue, signature enforceability, tax correctness or zero possible data loss is authorized by this PRD.

36 Handover summary and traceability

The release succeeds when a real owner can record and publish a quote, a real customer can approve the exact version, the owner can capture and obtain approval for a change, and the final invoice and ledger remain correct through interruptions, corrections, subscription changes and account lifecycle events. Every included feature serves that complete journey.

Requirement family	Primary screens	Required evidence
ACC AUTHZ CUS CAT	S01 to S07 S19 S20 S22	Identity/isolation and contact snapshot tests
JOB QUO CHG APR	S08 to S14 S25 to S27	Approval/version/concurrency end-to-end tests
FIN BIL	S15 to S18	F01 to F12 plus financial property tests
SUB	S21	Real sandbox store event and restore cases
SYNC DOC EXP	S09 S11 S23 S24	Physical device interruption, PDF/export integrity
SEC PRV	S22 S24 S25 S28	Security review, deletion and access-control evidence
NFR ANA OPS	All	Load/accessibility/telemetry/restore results
DEL REL	Release package	Owner-controlled source/assets/configuration and completed checklist

A developer handover is incomplete if it provides only a working-looking interface. It must include the data integrity, operational recovery, customer verification, subscription handling and test evidence specified here. Product, engineering, QA and operations leads record acceptance against the same version of this document before the public launch.